ML Bootcamp — Evaluation 1 Review

Summer 2026 · KIAT · University of Toronto

Jinyoung Ko

July 10, 2026

Q3 — Standardization

Let's say the 'age' feature is standardized with `StandardScaler` using $z = (x - \mu)/\sigma$. Suppose 'age' has mean $\mu = 30$ and standard deviation $\sigma = 8$. What is the standardized value of a passenger whose age is 46?

A. 16

B. -2

C. 2

D. 0.5

Q3 — Standardization — Answer

Review

Feature scaling puts features on comparable ranges. The two workhorses: **standardization** (z-score), $z = (x - \mu)/\sigma$ — mean 0, std 1, reads as "how many standard deviations from the mean"; and **min–max normalization**, $x' = (x - x_{\min})/(x_{\max} - x_{\min})$ — squashes into $[0, 1]$ but is sensitive to outliers. (Robust scaling, median/IQR, is the outlier-resistant third option.) Scaling matters most for distance- and gradient-based models: k-NN, SVM, PCA, neural networks.

Q3 — Standardization — Answer

Review

Feature scaling puts features on comparable ranges. The two workhorses: **standardization** (z-score), $z = (x - \mu)/\sigma$ — mean 0, std 1, reads as "how many standard deviations from the mean"; and **min–max normalization**, $x' = (x - x_{\min})/(x_{\max} - x_{\min})$ — squashes into $[0, 1]$ but is sensitive to outliers. (Robust scaling, median/IQR, is the outlier-resistant third option.) Scaling matters most for distance- and gradient-based models: k-NN, SVM, PCA, neural networks.

✓ **Answer: C.** $z = (x - \mu)/\sigma = (46 - 30)/8 = \mathbf{2}$ — this passenger is two standard deviations above the mean age.

Q4 — One-hot vs. label encoding

The nominal variable 'color' can be encoded as blue$=0$, green$=1$, red$=2$, or can be one-hot encoded into three $0/1$ columns (e.g., $[0, 0, 1]$). For a model that treats its inputs numerically, why is one-hot encoding usually preferred over label encoding for a variable like 'color'?

A. One-hot encoding uses fewer columns than label encoding, so it trains faster and uses less memory.

B. The integers 0, 1, 2 imply an ordering and spacing among the colors that is not real, so a numeric model may treat red as greater than blue.

C. Label encoding can only encode two categories at a time, so three colors cannot be represented.

D. One-hot encoding replaces each color with how frequently it appears, which carries more information.

Q4 — One-hot vs. label encoding — Answer

Review

Label encoding is for *ordinal* variables, where order is meaningful (small $<$ medium $<$ large); 'color' is *nominal*, so its categories have no order to encode.

Q4 — One-hot vs. label encoding — Answer

Review

Label encoding is for *ordinal* variables, where order is meaningful (small $<$ medium $<$ large); 'color' is *nominal*, so its categories have no order to encode.

A. One-hot encoding uses fewer columns than label encoding, so it trains faster and uses less memory.

× Backwards — one-hot uses *more* columns (one per category), not fewer.

Q4 — One-hot vs. label encoding — Answer

Review

Label encoding is for *ordinal* variables, where order is meaningful (small $<$ medium $<$ large); 'color' is *nominal*, so its categories have no order to encode.

A. One-hot encoding uses fewer columns than label encoding, so it trains faster and uses less memory.

× Backwards — one-hot uses *more* columns (one per category), not fewer.

B. The integers 0, 1, 2 imply an ordering and spacing among the colors that is not real, so a numeric model may treat red as greater than blue.

✓ The integers fake an order and spacing; a numeric model will exploit "red $>$ blue".

Q4 — One-hot vs. label encoding — Answer

Review

Label encoding is for *ordinal* variables, where order is meaningful (small $<$ medium $<$ large); 'color' is *nominal*, so its categories have no order to encode.

A. One-hot encoding uses fewer columns than label encoding, so it trains faster and uses less memory.

× Backwards — one-hot uses *more* columns (one per category), not fewer.

B. The integers 0, 1, 2 imply an ordering and spacing among the colors that is not real, so a numeric model may treat red as greater than blue.

✓ The integers fake an order and spacing; a numeric model will exploit "red $>$ blue".

C. Label encoding can only encode two categories at a time, so three colors cannot be represented.

× `LabelEncoder` handles any number of categories; two is not a limit.

Q4 — One-hot vs. label encoding — Answer

Review

Label encoding is for *ordinal* variables, where order is meaningful (small $<$ medium $<$ large); 'color' is *nominal*, so its categories have no order to encode.

A. One-hot encoding uses fewer columns than label encoding, so it trains faster and uses less memory.

× Backwards — one-hot uses *more* columns (one per category), not fewer.

B. The integers 0, 1, 2 imply an ordering and spacing among the colors that is not real, so a numeric model may treat red as greater than blue.

✓ The integers fake an order and spacing; a numeric model will exploit "red $>$ blue".

C. Label encoding can only encode two categories at a time, so three colors cannot be represented.

× `LabelEncoder` handles any number of categories; two is not a limit.

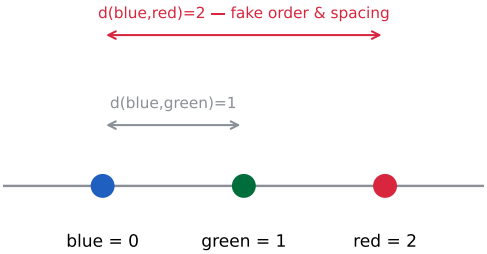
D. One-hot encoding replaces each color with how frequently it appears, which carries more information.

× That describes *frequency encoding* — a different technique.

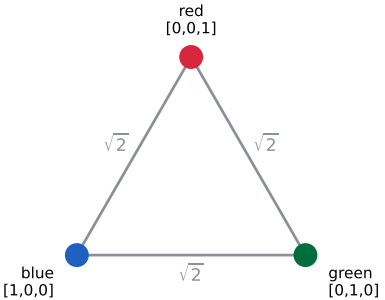
Demo: label encoding invents distances, one-hot doesn't

One-hot keeps all categories equally far apart

label encoding



one-hot encoding



Q5 — Outlier visualization

Which of the following plots is good at showing the outliers?

A. Box Plot

B. Pie Chart

C. Line Chart

D. Heatmaps

Q5 — Outlier visualization — Answer

Review

Every chart answers a different question — pick the plot by the question you are asking of the data; “are there extreme values?” is a *distribution* question.

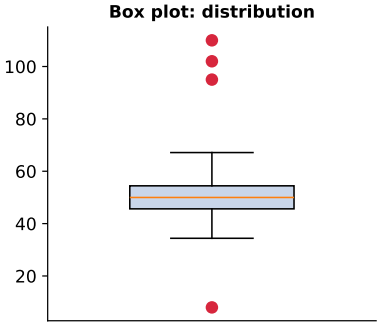
Q5 — Outlier visualization — Answer

Review

Every chart answers a different question — pick the plot by the question you are asking of the data; "are there extreme values?" is a *distribution* question.

A. Box Plot

✓ Points beyond the whiskers are drawn individually — outliers are literally visible dots.



Q5 — Outlier visualization — Answer

Review

Every chart answers a different question — pick the plot by the question you are asking of the data; "are there extreme values?" is a *distribution* question.

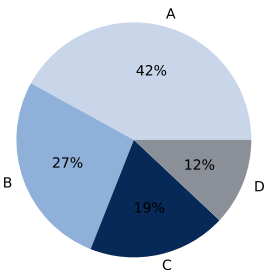
A. Box Plot

✓ Points beyond the whiskers are drawn individually — outliers are literally visible dots.

B. Pie Chart

× Pie charts show composition (shares of a whole) — apply them to category proportions.

Pie chart: composition



Q5 — Outlier visualization — Answer

Review

Every chart answers a different question — pick the plot by the question you are asking of the data; "are there extreme values?" is a *distribution* question.

A. Box Plot

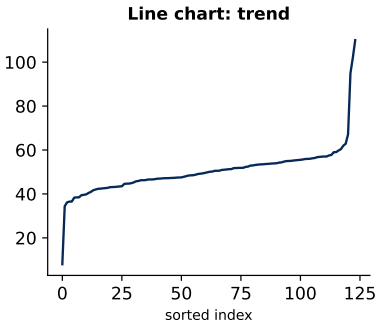
✓ Points beyond the whiskers are drawn individually — outliers are literally visible dots.

B. Pie Chart

× Pie charts show composition (shares of a whole) — apply them to category proportions.

C. Line Chart

× Line charts show trends over an ordered axis — apply them to time series.



Q5 — Outlier visualization — Answer

Review

Every chart answers a different question — pick the plot by the question you are asking of the data; "are there extreme values?" is a *distribution* question.

A. Box Plot

✓ Points beyond the whiskers are drawn individually — outliers are literally visible dots.

B. Pie Chart

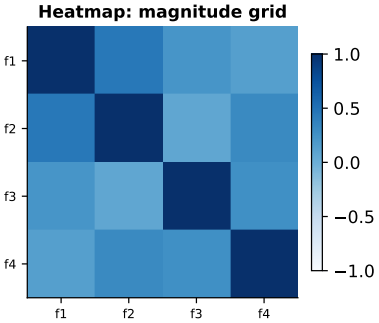
× Pie charts show composition (shares of a whole) — apply them to category proportions.

C. Line Chart

× Line charts show trends over an ordered axis — apply them to time series.

D. Heatmaps

× Heatmaps show magnitude over a grid — apply them to correlation matrices.



Q6 — Data cleaning objectives

What of the following is **not** an objective of data cleaning?

A. Creating new data records.

B. Removing duplicate records.

C. Handling missing data.

D. Correcting inaccurate values

Q6 — Data cleaning objectives — Answer

A. Creating new data records.

✓ Creating records is data *generation/augmentation* — a different step with different risks.

Q6 — Data cleaning objectives — Answer

A. Creating new data records.

✓ Creating records is data *generation/augmentation* — a different step with different risks.

B. Removing duplicate records.

× Removing duplicates IS core data cleaning.

Q6 — Data cleaning objectives — Answer

A. Creating new data records.

✓ Creating records is data *generation/augmentation* — a different step with different risks.

B. Removing duplicate records.

× Removing duplicates IS core data cleaning.

C. Handling missing data.

× Handling missing data IS core data cleaning (imputation or removal).

Q6 — Data cleaning objectives — Answer

A. Creating new data records.

✓ Creating records is data *generation/augmentation* — a different step with different risks.

B. Removing duplicate records.

× Removing duplicates IS core data cleaning.

C. Handling missing data.

× Handling missing data IS core data cleaning (imputation or removal).

D. Correcting inaccurate values

× Correcting wrong values IS core data cleaning.

Q7 — Feature selection

Why do we use feature selection in machine learning?

A. To change data into a machine learning format.

B. To pick the best machine learning method.

C. To choose the important features.

D. To fill in missing data.

Q7 — Feature selection — Answer

Review

The ML pipeline has distinct steps: preprocess the data, select the features, select the model, then train and evaluate — each option here names a different step.

Q7 — Feature selection — Answer

Review

The ML pipeline has distinct steps: preprocess the data, select the features, select the model, then train and evaluate — each option here names a different step.

A. To change data into a machine learning format.

× Changing data format is *preprocessing* (encoding, scaling).

Q7 — Feature selection — Answer

Review

The ML pipeline has distinct steps: preprocess the data, select the features, select the model, then train and evaluate — each option here names a different step.

A. To change data into a machine learning format.

× Changing data format is *preprocessing* (encoding, scaling).

B. To pick the best machine learning method.

× Picking the method is *model selection*, not feature selection.

Q7 — Feature selection — Answer

Review

The ML pipeline has distinct steps: preprocess the data, select the features, select the model, then train and evaluate — each option here names a different step.

A. To change data into a machine learning format.

× Changing data format is *preprocessing* (encoding, scaling).

B. To pick the best machine learning method.

× Picking the method is *model selection*, not feature selection.

C. To choose the important features.

✓ Keep the informative columns, drop the rest: less overfitting, faster training.

Q7 — Feature selection — Answer

Review

The ML pipeline has distinct steps: preprocess the data, select the features, select the model, then train and evaluate — each option here names a different step.

A. To change data into a machine learning format.

× Changing data format is *preprocessing* (encoding, scaling).

B. To pick the best machine learning method.

× Picking the method is *model selection*, not feature selection.

C. To choose the important features.

✓ Keep the informative columns, drop the rest: less overfitting, faster training.

D. To fill in missing data.

× Filling missing values is *imputation* — a preprocessing step.

Q8 — Bagging vs. boosting

You've trained a deep decision tree on your dataset and noticed that it performs really well on the training data but poorly on the validation data. Which ensemble technique might help remedy this, bagging or boosting?

A. Bagging

B. Boosting

Q8 — Bagging vs. boosting — Answer

Review

Model error splits into **bias** (model too simple: wrong on training and validation alike) and **variance** (model too flexible: great on training, poor on validation) — diagnose which you have first; the right ensemble follows.

Q8 — Bagging vs. boosting — Answer

Review

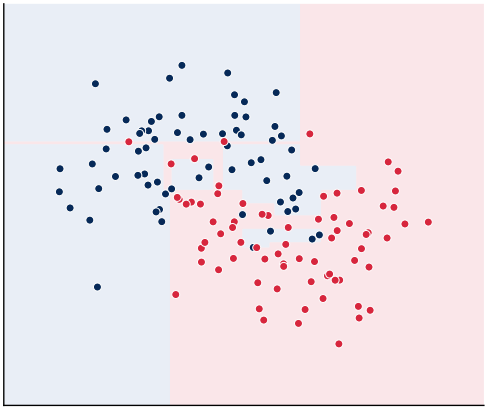
Model error splits into **bias** (model too simple: wrong on training and validation alike) and **variance** (model too flexible: great on training, poor on validation) — diagnose which you have first; the right ensemble follows.

✓ **Answer: A. Bagging.** Great-on-train / poor-on-validation is the *variance* signature, so average many bootstrap trees. Boosting targets *bias* — the underfitting failure — and can amplify overfitting on an already-deep tree.

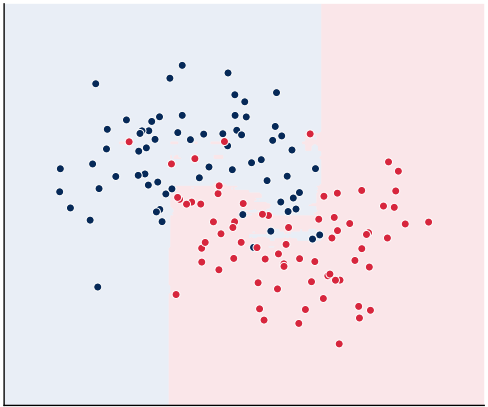
Demo: one deep tree vs. a bag of 100 trees

Bagging averages away the variance of individual deep trees

1 deep tree — train 100% / test 85%



bag of 100 trees — train 100% / test 92%



Q9 — Purpose of regularization

What is regularization for?

A. To speed up model training by reducing the number of computations.

B. To discourage overly complex models so they generalize better to unseen data.

C. To fill in missing values before the model is trained.

D. To increase the model's accuracy on the training set.

Q9 — Purpose of regularization — Answer

A. To speed up model training by reducing the number of computations.

× Regularization *adds* computation if anything; speed is not the goal.

Q9 — Purpose of regularization — Answer

A. To speed up model training by reducing the number of computations.

× Regularization *adds* computation if anything; speed is not the goal.

B. To discourage overly complex models so they generalize better to unseen data.

✓ Penalizing complexity pushes the model toward solutions that generalize.

Q9 — Purpose of regularization — Answer

A. To speed up model training by reducing the number of computations.

× Regularization *adds* computation if anything; speed is not the goal.

B. To discourage overly complex models so they generalize better to unseen data.

✓ Penalizing complexity pushes the model toward solutions that generalize.

C. To fill in missing values before the model is trained.

× Filling missing values is imputation — preprocessing, not regularization.

Q9 — Purpose of regularization — Answer

A. To speed up model training by reducing the number of computations.

× Regularization *adds* computation if anything; speed is not the goal.

B. To discourage overly complex models so they generalize better to unseen data.

✓ Penalizing complexity pushes the model toward solutions that generalize.

C. To fill in missing values before the model is trained.

× Filling missing values is imputation — preprocessing, not regularization.

D. To increase the model's accuracy on the training set.

× Backwards: regularization usually makes *training* accuracy worse. That sacrifice is the point.

Q10 — What is NOT regularization

Which of the following is **NOT** intended to regularize an ML model?

A. Dropout in a neural network.

B. Adding an L2 penalty on the weights.

C. Standardizing features to zero mean and unit variance.

D. Pruning a decision tree (or capping `max_depth`)

Q10 — What is NOT regularization — Answer

Review

Ask of any technique: does it *restrict what the model can express*? If yes, it is regularization. If it merely transforms the inputs for comparability or numerical stability, it is preprocessing — useful, sometimes essential, but not a constraint on model complexity.

Q10 — What is NOT regularization — Answer

Review

Ask of any technique: does it *restrict what the model can express*? If yes, it is regularization. If it merely transforms the inputs for comparability or numerical stability, it is preprocessing — useful, sometimes essential, but not a constraint on model complexity.

A. Dropout in a neural network.

× Dropout IS a regularizer — randomly disabling units during training limits co-adaptation.

Q10 — What is NOT regularization — Answer

Review

Ask of any technique: does it *restrict what the model can express*? If yes, it is regularization. If it merely transforms the inputs for comparability or numerical stability, it is preprocessing — useful, sometimes essential, but not a constraint on model complexity.

A. Dropout in a neural network.

× Dropout IS a regularizer — randomly disabling units during training limits co-adaptation.

B. Adding an L2 penalty on the weights.

× An L2 penalty IS a regularizer — it shrinks the weights.

Q10 — What is NOT regularization — Answer

Review

Ask of any technique: does it *restrict what the model can express*? If yes, it is regularization. If it merely transforms the inputs for comparability or numerical stability, it is preprocessing — useful, sometimes essential, but not a constraint on model complexity.

A. Dropout in a neural network.

× Dropout IS a regularizer — randomly disabling units during training limits co-adaptation.

B. Adding an L2 penalty on the weights.

× An L2 penalty IS a regularizer — it shrinks the weights.

C. Standardizing features to zero mean and unit variance.

✓ Standardization is *preprocessing*: it makes features comparable (vital for k-NN, SVM, PCA, gradient descent) but does not constrain model complexity.

Q10 — What is NOT regularization — Answer

Review

Ask of any technique: does it *restrict what the model can express*? If yes, it is regularization. If it merely transforms the inputs for comparability or numerical stability, it is preprocessing — useful, sometimes essential, but not a constraint on model complexity.

A. Dropout in a neural network.

× Dropout IS a regularizer — randomly disabling units during training limits co-adaptation.

B. Adding an L2 penalty on the weights.

× An L2 penalty IS a regularizer — it shrinks the weights.

C. Standardizing features to zero mean and unit variance.

✓ Standardization is *preprocessing*: it makes features comparable (vital for k-NN, SVM, PCA, gradient descent) but does not constrain model complexity.

D. Pruning a decision tree (or capping `max_depth`)

× Pruning / capping `max_depth` IS a regularizer — it directly caps tree complexity.

Q11 — k-NN classification

A new point is at $(5, 5)$. Using Euclidean distance and $k = 3$, which class does k-NN assign it to?

Label A data points: $(4, 5)$, $(2, 2)$, $(1, 5)$, $(3, 8)$
Label B data points: $(6, 6)$, $(6, 4)$, $(7, 5)$, $(8, 8)$

A. A

B. B

Q11 — k-NN classification — Answer

Review

k-NN assigns the label by majority vote among the k nearest points — the distance metric and k are choices you make, not things the model learns.

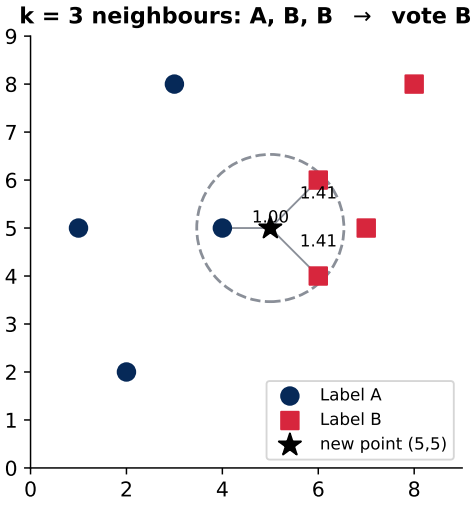
Q11 — k-NN classification — Answer

Review

k-NN assigns the label by majority vote among the k nearest points — the distance metric and k are choices you make, not things the model learns.

✓ **Answer: B.** Distances from $(5,5)$: $(4,5) \to 1.0$ (A), $(6,4) \to 1.41$ (B), $(6,6) \to 1.41$ (B) — the vote is 2–1 for B. With $k = 1$ the answer would be A: k is a hyperparameter, and it changes the result.

Demo: the three nearest neighbours of (5,5)



Q12 — Hyperparameters

Among those what is **not** hyperparameters?

A. The number of neurons in one layer of neural network

B. Learning rate in gradient descent

C. The coefficient of regularization term

D. The threshold value chosen at each node of a decision tree

Q12 — Hyperparameters — Answer

Review

A **hyperparameter** is set *before* training; a **parameter** is *found by* training — ask which side of the fit each quantity lives on.

Q12 — Hyperparameters — Answer

Review

A **hyperparameter** is set *before* training; a **parameter** is *found by* training — ask which side of the fit each quantity lives on.

A. The number of neurons in one layer of neural network

× A hyperparameter — you choose the layer width before training.

Q12 — Hyperparameters — Answer

Review

A **hyperparameter** is set *before* training; a **parameter** is *found by* training — ask which side of the fit each quantity lives on.

A. The number of neurons in one layer of neural network

× A hyperparameter — you choose the layer width before training.

B. Learning rate in gradient descent

× A hyperparameter — you set the step size before training.

Q12 — Hyperparameters — Answer

Review

A **hyperparameter** is set *before* training; a **parameter** is *found by* training — ask which side of the fit each quantity lives on.

A. The number of neurons in one layer of neural network

× A hyperparameter — you choose the layer width before training.

B. Learning rate in gradient descent

× A hyperparameter — you set the step size before training.

C. The coefficient of regularization term

× A hyperparameter — λ is tuned by cross-validation, never learned by the fit itself.

Q12 — Hyperparameters — Answer

Review

A **hyperparameter** is set *before* training; a **parameter** is *found by* training — ask which side of the fit each quantity lives on.

A. The number of neurons in one layer of neural network

× A hyperparameter — you choose the layer width before training.

B. Learning rate in gradient descent

× A hyperparameter — you set the step size before training.

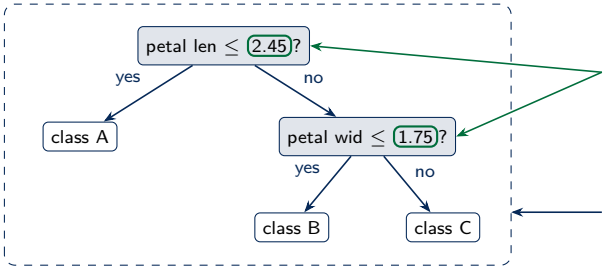
C. The coefficient of regularization term

× A hyperparameter — λ is tuned by cross-validation, never learned by the fit itself.

D. The threshold value chosen at each node of a decision tree

✓ *Learned*: the fitting algorithm finds each node's threshold from the data — like the weights of a neural network.

Demo: parameters vs. hyperparameters on one tree



thresholds found **by** training
⇒ **parameters**

`max_depth=2, criterion='gini'`
set **before** training ⇒ **hyperparameters**

Q13 — Recall

A bank subscription classifier is evaluated on 100 clients. Of the 50 clients who actually subscribed (the positive class), the model correctly flagged 40 and missed 10. Of the 50 who did not subscribe, it wrongly flagged 20 and correctly cleared 30. What is the model's RECALL on the positive class?

A. 0.67

B. 0.7

C. 0.6

D. 0.80

Q13 — Recall — Answer

Review

Recall answers: of everyone who is *actually* positive, what fraction did we catch? Precision, accuracy, and specificity answer different questions on the same confusion matrix.

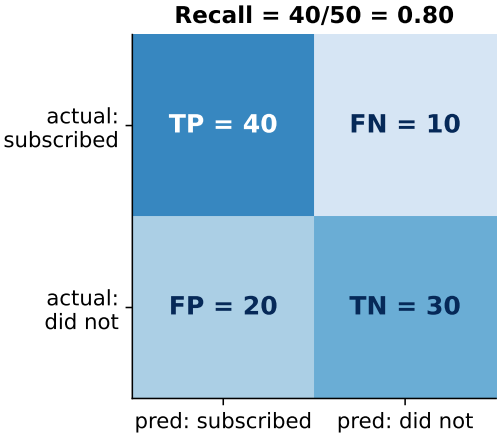
Q13 — Recall — Answer

Review

Recall answers: of everyone who is *actually* positive, what fraction did we catch? Precision, accuracy, and specificity answer different questions on the same confusion matrix.

✓ **Answer: D.** Recall $= \mathrm{TP}/(\mathrm{TP}+\mathrm{FN}) = 40/50 = \mathbf{0.80}$. Every distractor is a *real* metric on this same table: 0.67 = precision, 0.70 = accuracy, 0.60 = specificity (see the demo).

Demo: one confusion matrix, four metrics




```
recall    = TP/(TP+FN) = 40/50 = 0.80
precision = TP/(TP+FP) = 40/60 = 0.67
accuracy  = (TP+TN)/100 = 0.70
specificity = TN/(TN+FP) = 30/50 = 0.60
```

Q14 — Hyperparameter tuning

Which technique is commonly used for finding the best hyperparameter tuning

A. Feature Enginnering

B. Cross-Validation

C. Data Preprocessing

D. Model Inference

Q14 — Hyperparameter tuning — Answer

A. Feature Enginnering

× Feature engineering creates better inputs — valuable, but it does not compare hyperparameter settings.

Q14 — Hyperparameter tuning — Answer

A. Feature Enginnering

× Feature engineering creates better inputs — valuable, but it does not compare hyperparameter settings.

B. Cross-Validation

✓ Cross-validation gives each setting an honest score — `GridSearchCV(..., cv=5)` is exactly this.

Q14 — Hyperparameter tuning — Answer

A. Feature Enginnering

× Feature engineering creates better inputs — valuable, but it does not compare hyperparameter settings.

B. Cross-Validation

✓ Cross-validation gives each setting an honest score — `GridSearchCV(..., cv=5)` is exactly this.

C. Data Preprocessing

× Preprocessing cleans and scales inputs; it does not choose hyperparameters.

Q14 — Hyperparameter tuning — Answer

A. Feature Enginnering

× Feature engineering creates better inputs — valuable, but it does not compare hyperparameter settings.

B. Cross-Validation

✓ Cross-validation gives each setting an honest score — `GridSearchCV(..., cv=5)` is exactly this.

C. Data Preprocessing

× Preprocessing cleans and scales inputs; it does not choose hyperparameters.

D. Model Inference

× Inference is *using* the trained model — tuning happens before that.

Q15 — Why cross-validation

Why cross-validation is more reliable than a single split?

A. The model trains on all of the data every time, so no points are ever held out and no luck enters the estimate.

B. Each example is used for validation exactly once and the five fold scores are averaged, so the estimate depends far less on one lucky or unlucky split.

C. It guarantees the tuned model cannot overfit, because averaging over five folds cancels out the variance of the model's learned parameters.

D. Each of the five fits is validated on the four training folds combined, giving every fit a larger and therefore lower-variance test set.

Q15 — Why cross-validation — Answer

A. The model trains on all of the data every time, so no points are ever held out and no luck enters the estimate.

× False — CV always holds out one fold per fit; the model never trains on everything at once.

Q15 — Why cross-validation — Answer

A. The model trains on all of the data every time, so no points are ever held out and no luck enters the estimate.

× False — CV always holds out one fold per fit; the model never trains on everything at once.

B. Each example is used for validation exactly once and the five fold scores are averaged, so the estimate depends far less on one lucky or unlucky split.

✓ Every example is validated exactly once and the fold scores are averaged — far less dependence on one lucky split.

Q15 — Why cross-validation — Answer

A. The model trains on all of the data every time, so no points are ever held out and no luck enters the estimate.

× False — CV always holds out one fold per fit; the model never trains on everything at once.

B. Each example is used for validation exactly once and the five fold scores are averaged, so the estimate depends far less on one lucky or unlucky split.

✓ Every example is validated exactly once and the fold scores are averaged — far less dependence on one lucky split.

C. It guarantees the tuned model cannot overfit, because averaging over five folds cancels out the variance of the model's learned parameters.

× Nothing *guarantees* no overfitting; CV only measures generalization more reliably.

Q15 — Why cross-validation — Answer

A. The model trains on all of the data every time, so no points are ever held out and no luck enters the estimate.

× False — CV always holds out one fold per fit; the model never trains on everything at once.

B. Each example is used for validation exactly once and the five fold scores are averaged, so the estimate depends far less on one lucky or unlucky split.

✓ Every example is validated exactly once and the fold scores are averaged — far less dependence on one lucky split.

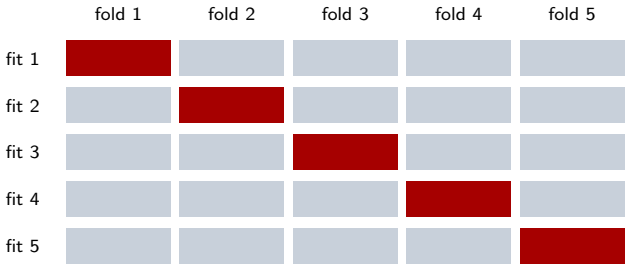
C. It guarantees the tuned model cannot overfit, because averaging over five folds cancels out the variance of the model's learned parameters.

× Nothing *guarantees* no overfitting; CV only measures generalization more reliably.

D. Each of the five fits is validated on the four training folds combined, giving every fit a larger and therefore lower-variance test set.

× Validation happens on the *held-out* fold — not on the four training folds.

Demo: 5-fold cross-validation



Q16 — Lasso feature selection

You want the final model to use only a handful of those features (automatic feature selection). Which regularization approach do you want to use?

A. Ridge (L2), because squaring the coefficients forces the unimportant ones exactly to zero.

B. Lasso (L1), because it shrinks every coefficient by the same small fraction, keeping all 21 features but making them tiny.

C. Lasso (L1), because its absolute-value penalty pushes weak coefficients exactly to zero, dropping those features from the model.

D. Ridge (L2), because it keeps only the coefficients with the largest absolute values and zeros out the rest.

Q16 — Lasso feature selection — Answer

Review

The course notebook says it in one line: *"L1 zeros them out, L2 just shrinks them."*

Q16 — Lasso feature selection — Answer

Review

The course notebook says it in one line: *"L1 zeros them out, L2 just shrinks them."*

A. Ridge (L2), because squaring the coefficients forces the unimportant ones exactly to zero.

× Ridge's squared penalty shrinks smoothly but (almost) never snaps a weight to exactly zero.

Q16 — Lasso feature selection — Answer

Review

The course notebook says it in one line: *"L1 zeros them out, L2 just shrinks them."*

A. Ridge (L2), because squaring the coefficients forces the unimportant ones exactly to zero.

× Ridge's squared penalty shrinks smoothly but (almost) never snaps a weight to exactly zero.

B. Lasso (L1), because it shrinks every coefficient by the same small fraction, keeping all 21 features but making them tiny.

× Names Lasso but *describes Ridge*: proportional shrinkage that keeps every feature.

Q16 — Lasso feature selection — Answer

Review

The course notebook says it in one line: *"L1 zeros them out, L2 just shrinks them."*

A. Ridge (L2), because squaring the coefficients forces the unimportant ones exactly to zero.

× Ridge's squared penalty shrinks smoothly but (almost) never snaps a weight to exactly zero.

B. Lasso (L1), because it shrinks every coefficient by the same small fraction, keeping all 21 features but making them tiny.

× Names Lasso but *describes Ridge*: proportional shrinkage that keeps every feature.

C. Lasso (L1), because its absolute-value penalty pushes weak coefficients exactly to zero, dropping those features from the model.

✓ The $|w|$ penalty keeps constant slope near zero, so weak coefficients hit exactly 0 and their features drop out.

Q16 — Lasso feature selection — Answer

Review

The course notebook says it in one line: *"L1 zeros them out, L2 just shrinks them."*

A. Ridge (L2), because squaring the coefficients forces the unimportant ones exactly to zero.

× Ridge's squared penalty shrinks smoothly but (almost) never snaps a weight to exactly zero.

B. Lasso (L1), because it shrinks every coefficient by the same small fraction, keeping all 21 features but making them tiny.

× Names Lasso but *describes Ridge*: proportional shrinkage that keeps every feature.

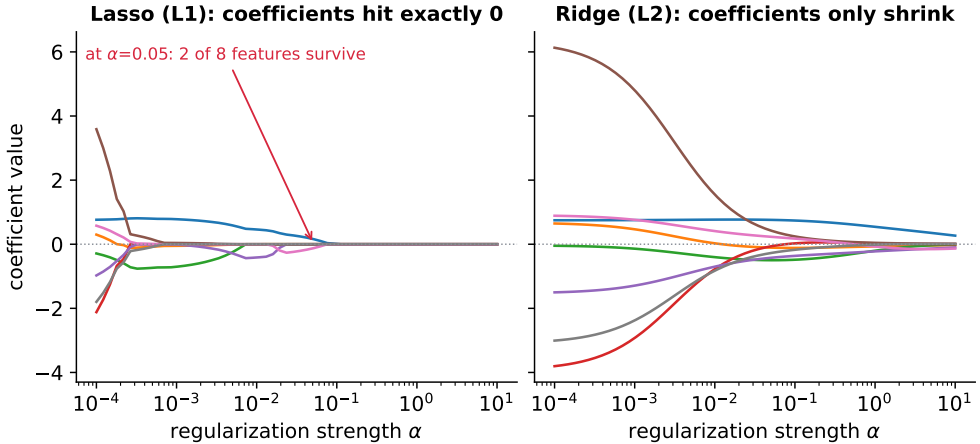
C. Lasso (L1), because its absolute-value penalty pushes weak coefficients exactly to zero, dropping those features from the model.

✓ The $|w|$ penalty keeps constant slope near zero, so weak coefficients hit exactly 0 and their features drop out.

D. Ridge (L2), because it keeps only the coefficients with the largest absolute values and zeros out the rest.

× Ridge has no zeroing mechanism at all — "keep the largest, zero the rest" describes neither method.

Demo: L1 zeros coefficients out, L2 only shrinks them



Q17 — k-means++

What problem is k-means++ designed to fix over ordinary k-means?

A. k-means++ mathematically guarantees the lowest possible WCSS on any dataset, no matter how the individual points happen to be arranged.

B. Random starts leave the features on different scales, so k-means++ first standardizes every feature to zero mean and unit variance.

C. k-means++ removes the need to fix k beforehand by adding centroids until the within-cluster sum of squares reaches zero.

D. Purely random starts can place two centroids almost on top of each other, which slows convergence and risks a poor final clustering.

Q17 — k-means++ — Answer

Review

k-means always converges — but only to a *local* optimum determined by where the k centroids start, so initialization is a quality lever, not a detail.

Q17 — k-means++ — Answer

Review

k-means always converges — but only to a *local* optimum determined by where the k centroids start, so initialization is a quality lever, not a detail.

A. k-means++ mathematically guarantees the lowest possible WCSS on any dataset, no matter how the individual points happen to be arranged.

× No initialization scheme can guarantee the global WCSS optimum — k-means still converges locally.

Q17 — k-means++ — Answer

Review

k-means always converges — but only to a *local* optimum determined by where the k centroids start, so initialization is a quality lever, not a detail.

A. k-means++ mathematically guarantees the lowest possible WCSS on any dataset, no matter how the individual points happen to be arranged.

× No initialization scheme can guarantee the global WCSS optimum — k-means still converges locally.

B. Random starts leave the features on different scales, so k-means++ first standardizes every feature to zero mean and unit variance.

× Feature scaling is a separate preprocessing step; k-means++ does not standardize anything.

Q17 — k-means++ — Answer

Review

k-means always converges — but only to a *local* optimum determined by where the k centroids start, so initialization is a quality lever, not a detail.

A. k-means++ mathematically guarantees the lowest possible WCSS on any dataset, no matter how the individual points happen to be arranged.

× No initialization scheme can guarantee the global WCSS optimum — k-means still converges locally.

B. Random starts leave the features on different scales, so k-means++ first standardizes every feature to zero mean and unit variance.

× Feature scaling is a separate preprocessing step; k-means++ does not standardize anything.

C. k-means++ removes the need to fix k beforehand by adding centroids until the within-cluster sum of squares reaches zero.

× k is still fixed in advance — k-means++ only chooses *where* the k starting centroids go.

Q17 — k-means++ — Answer

Review

k-means always converges — but only to a *local* optimum determined by where the k centroids start, so initialization is a quality lever, not a detail.

A. k-means++ mathematically guarantees the lowest possible WCSS on any dataset, no matter how the individual points happen to be arranged.

× No initialization scheme can guarantee the global WCSS optimum — k-means still converges locally.

B. Random starts leave the features on different scales, so k-means++ first standardizes every feature to zero mean and unit variance.

× Feature scaling is a separate preprocessing step; k-means++ does not standardize anything.

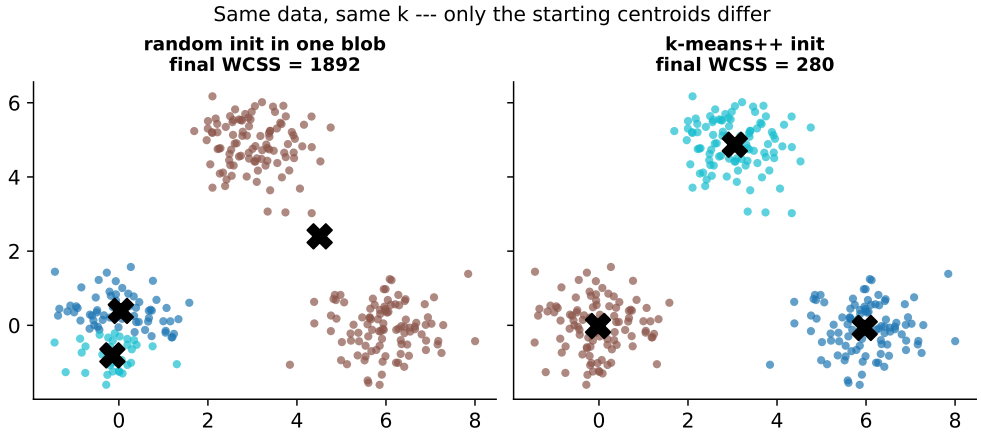
C. k-means++ removes the need to fix k beforehand by adding centroids until the within-cluster sum of squares reaches zero.

× k is still fixed in advance — k-means++ only chooses *where* the k starting centroids go.

D. Purely random starts can place two centroids almost on top of each other, which slows convergence and risks a poor final clustering.

✓ Uniformly random starts can drop two centroids into one blob; ++ spreads them by squared distance.

Demo: why initialization matters in k-means



Q18 — Classification metrics

Which of the following is **not** typically considered as an evaluation metrics for classification models?

A. Precision

B. Recall

C. Feature Importance

D. F1-Score

Q18 — Classification metrics — Answer

Review

An evaluation metric scores *predictions against true labels* — anything that instead describes the model's internals is interpretation, not evaluation.

Q18 — Classification metrics — Answer

Review

An evaluation metric scores *predictions against true labels* — anything that instead describes the model's internals is interpretation, not evaluation.

A. Precision

× Precision IS an evaluation metric — correctness of the positive flags.

Q18 — Classification metrics — Answer

Review

An evaluation metric scores *predictions against true labels* — anything that instead describes the model's internals is interpretation, not evaluation.

A. Precision

× Precision IS an evaluation metric — correctness of the positive flags.

B. Recall

× Recall IS an evaluation metric — coverage of the true positives.

Q18 — Classification metrics — Answer

Review

An evaluation metric scores *predictions against true labels* — anything that instead describes the model's internals is interpretation, not evaluation.

A. Precision

× Precision IS an evaluation metric — correctness of the positive flags.

B. Recall

× Recall IS an evaluation metric — coverage of the true positives.

C. Feature Importance

✓ Feature importance describes the *model's internals* (which inputs it leaned on) — interpretation, not prediction quality.

Q18 — Classification metrics — Answer

Review

An evaluation metric scores *predictions against true labels* — anything that instead describes the model's internals is interpretation, not evaluation.

A. Precision

× Precision IS an evaluation metric — correctness of the positive flags.

B. Recall

× Recall IS an evaluation metric — coverage of the true positives.

C. Feature Importance

✓ Feature importance describes the *model's internals* (which inputs it leaned on) — interpretation, not prediction quality.

D. F1-Score

× F1 IS an evaluation metric — the harmonic mean of precision and recall.

Q19 — Types of supervised learning

What are the two common types of supervised learning? (choose 2)

A. Classification

B. Clustering

C. Regression

Q19 — Types of supervised learning — Answer

✓ **Answer: Classification + Regression.** Supervised learning predicts a known target: a discrete label (classification) or a continuous value (regression). Clustering has no labels to supervise with — unsupervised.

Q20 — Supervised learning

Which of the following statements is true about supervised learning?

A. It is a type of machine learning where the model is trained on labeled data.

B. It is a type of machine learning where the model does not require any training data.

C. It is a type of machine learning where a human must intervene during the training process.

D. It is a type of machine learning that does not require labeled data.

Q20 — Supervised learning — Answer

A. It is a type of machine learning where the model is trained on labeled data.

✓ "Supervised" means every training example carries its correct label.

Q20 — Supervised learning — Answer

A. It is a type of machine learning where the model is trained on labeled data.

✓ "Supervised" means every training example carries its correct label.

B. It is a type of machine learning where the model does not require any training data.

× Every ML model needs training data — no exceptions.

Q20 — Supervised learning — Answer

A. It is a type of machine learning where the model is trained on labeled data.

✓ "Supervised" means every training example carries its correct label.

B. It is a type of machine learning where the model does not require any training data.

× Every ML model needs training data — no exceptions.

C. It is a type of machine learning where a human must intervene during the training process.

× The supervision is in the *labels*, not a human watching the training run.

Q20 — Supervised learning — Answer

A. It is a type of machine learning where the model is trained on labeled data.

✓ "Supervised" means every training example carries its correct label.

B. It is a type of machine learning where the model does not require any training data.

× Every ML model needs training data — no exceptions.

C. It is a type of machine learning where a human must intervene during the training process.

× The supervision is in the *labels*, not a human watching the training run.

D. It is a type of machine learning that does not require labeled data.

× That describes *unsupervised* learning (clustering, PCA).

Q21 — Linear regression inputs

For linear regression, which of the following are the inputs (also known as features) that are fed into the model and which the model makes a prediction on?

$$\underbrace{\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}}_{Y} = \underbrace{\begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_n \end{bmatrix}}_{X} \underbrace{\begin{bmatrix} \beta_0 \\ \beta_1 \end{bmatrix}}_{\beta} + \underbrace{\begin{bmatrix} \varepsilon_1 \\ \varepsilon_2 \\ \vdots \\ \varepsilon_n \end{bmatrix}}_{\varepsilon} \qquad Y = X\beta + \varepsilon$$

A. x

B. β

C. y

D. ε

Q21 — Linear regression inputs — Answer

A. x

✓ X is the feature matrix — one row per example, plus the column of 1s for the intercept.

Q21 — Linear regression inputs — Answer

A. x

✓ X is the feature matrix — one row per example, plus the column of 1s for the intercept.

B. β

× β holds the coefficients the model *learns* — outputs of training, not inputs.

Q21 — Linear regression inputs — Answer

A. x

✓ X is the feature matrix — one row per example, plus the column of 1s for the intercept.

B. β

× β holds the coefficients the model *learns* — outputs of training, not inputs.

C. y

× y is the target the model predicts.

Q21 — Linear regression inputs — Answer

A. x

✓ X is the feature matrix — one row per example, plus the column of 1s for the intercept.

B. β

× β holds the coefficients the model *learns* — outputs of training, not inputs.

C. y

× y is the target the model predicts.

D. ε

× ε is the noise the line cannot explain — never fed to the model.

Q22 — What is a gradient

What is a gradient?

A. The rate of change of the model's accuracy during training.

B. A vector that points in the direction of maximum increase of a function (usually the loss function)

C. A type of data structure used to store multiple values of the same data type for model training

D. A measure of the angle between two intersecting vectors (usually the training dataset and the testing dataset)

Q22 — What is a gradient — Answer

A. The rate of change of the model's accuracy during training.

× Accuracy is not differentiated — the gradient is of the *loss* with respect to the weights.

Q22 — What is a gradient — Answer

A. The rate of change of the model's accuracy during training.

× Accuracy is not differentiated — the gradient is of the *loss* with respect to the weights.

B. A vector that points in the direction of maximum increase of a function (usually the loss function)

✓ $\nabla J(w)$ stacks all partial derivatives and points uphill — which is why descent steps the *opposite* way.

Q22 — What is a gradient — Answer

A. The rate of change of the model's accuracy during training.

× Accuracy is not differentiated — the gradient is of the *loss* with respect to the weights.

B. A vector that points in the direction of maximum increase of a function (usually the loss function)

✓ $\nabla J(w)$ stacks all partial derivatives and points uphill — which is why descent steps the *opposite* way.

C. A type of data structure used to store multiple values of the same data type for model training

× That describes an array (a data structure) — not a derivative.

Q22 — What is a gradient — Answer

A. The rate of change of the model's accuracy during training.

× Accuracy is not differentiated — the gradient is of the *loss* with respect to the weights.

B. A vector that points in the direction of maximum increase of a function (usually the loss function)

✓ $\nabla J(w)$ stacks all partial derivatives and points uphill — which is why descent steps the *opposite* way.

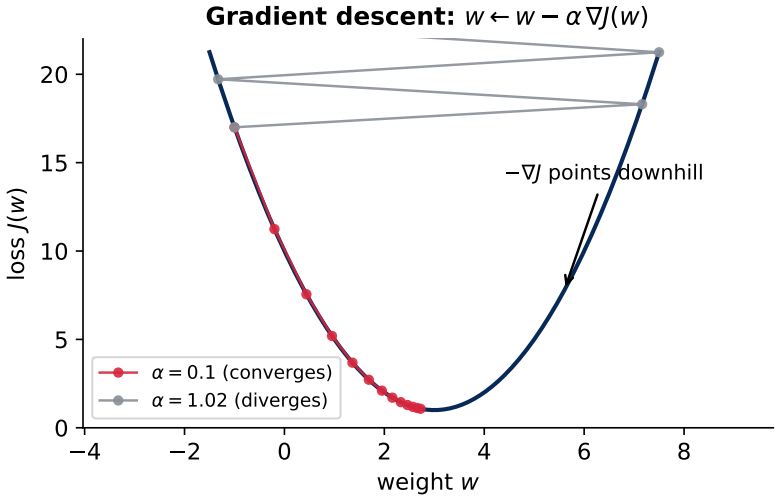
C. A type of data structure used to store multiple values of the same data type for model training

× That describes an array (a data structure) — not a derivative.

D. A measure of the angle between two intersecting vectors (usually the training dataset and the testing dataset)

× Angles between datasets are not a thing — distractor invented from "vector" vocabulary.

Demo: gradient descent and the learning rate



Q23 — Gradient descent

What is the purpose of gradient descent?

A. Visualizing data patterns

B. Generating random numbers to initialize a model

C. Minimizing the loss function to improve the model

D. Converting categorical data to numerical data

Q23 — Gradient descent — Answer

Review

Gradient descent fits models with continuous weights (logistic regression, neural networks); trees are built by greedy splitting instead, and Q53's linear regression even has a closed-form solution.

Q23 — Gradient descent — Answer

Review

Gradient descent fits models with continuous weights (logistic regression, neural networks); trees are built by greedy splitting instead, and Q53's linear regression even has a closed-form solution.

A. Visualizing data patterns

× Visualization is EDA — apply plotting libraries, not optimizers.

Q23 — Gradient descent — Answer

Review

Gradient descent fits models with continuous weights (logistic regression, neural networks); trees are built by greedy splitting instead, and Q53's linear regression even has a closed-form solution.

A. Visualizing data patterns

× Visualization is EDA — apply plotting libraries, not optimizers.

B. Generating random numbers to initialize a model

× Random initialization happens *once, before* gradient descent starts.

Q23 — Gradient descent — Answer

Review

Gradient descent fits models with continuous weights (logistic regression, neural networks); trees are built by greedy splitting instead, and Q53's linear regression even has a closed-form solution.

A. Visualizing data patterns

× Visualization is EDA — apply plotting libraries, not optimizers.

B. Generating random numbers to initialize a model

× Random initialization happens *once, before* gradient descent starts.

C. Minimizing the loss function to improve the model

✓ Measure the slope of the loss, step downhill, repeat — this is how continuous-weight models train.

Q23 — Gradient descent — Answer

Review

Gradient descent fits models with continuous weights (logistic regression, neural networks); trees are built by greedy splitting instead, and Q53's linear regression even has a closed-form solution.

A. Visualizing data patterns

× Visualization is EDA — apply plotting libraries, not optimizers.

B. Generating random numbers to initialize a model

× Random initialization happens *once, before* gradient descent starts.

C. Minimizing the loss function to improve the model

✓ Measure the slope of the loss, step downhill, repeat — this is how continuous-weight models train.

D. Converting categorical data to numerical data

× Converting categories to numbers is *encoding* — preprocessing.

Q24 — ReLU vs. sigmoid

Why do we use ReLU over sigmoid as an activation function?

A. Because ReLU outputs a value between 0 and 1, which is easier to interpret.

B. Because sigmoid can make gradient shrinks toward zero, slowing learning in deep networks.

C. Because ReLU is differentiable everywhere, while sigmoid is not.

D. Because ReLU guarantees the network will not overfit.

Q24 — ReLU vs. sigmoid — Answer

A. Because ReLU outputs a value between 0 and 1, which is easier to interpret.

× $(0, 1)$ output is *sigmoid's* property, and interpretability is not why activations are chosen.

Q24 — ReLU vs. sigmoid — Answer

A. Because ReLU outputs a value between 0 and 1, which is easier to interpret.

× $(0, 1)$ output is *sigmoid's* property, and interpretability is not why activations are chosen.

B. Because sigmoid can make gradient shrinks toward zero, slowing learning in deep networks.

✓ Sigmoid saturates for large $|z|$, its slope $\to 0$, and backprop multiplies those slopes across layers: the *vanishing gradient*. ReLU's slope stays 1 for $z > 0$.

Q24 — ReLU vs. sigmoid — Answer

A. Because ReLU outputs a value between 0 and 1, which is easier to interpret.

× $(0, 1)$ output is *sigmoid's* property, and interpretability is not why activations are chosen.

B. Because sigmoid can make gradient shrinks toward zero, slowing learning in deep networks.

✓ Sigmoid saturates for large $|z|$, its slope $\to 0$, and backprop multiplies those slopes across layers: the *vanishing gradient*. ReLU's slope stays 1 for $z > 0$.

C. Because ReLU is differentiable everywhere, while sigmoid is not.

× Backwards: sigmoid is smooth *everywhere*; ReLU is the one with a kink at $z = 0$.

Q24 — ReLU vs. sigmoid — Answer

A. Because ReLU outputs a value between 0 and 1, which is easier to interpret.

× $(0, 1)$ output is *sigmoid's* property, and interpretability is not why activations are chosen.

B. Because sigmoid can make gradient shrinks toward zero, slowing learning in deep networks.

✓ Sigmoid saturates for large $|z|$, its slope $\to 0$, and backprop multiplies those slopes across layers: the *vanishing gradient*. ReLU's slope stays 1 for $z > 0$.

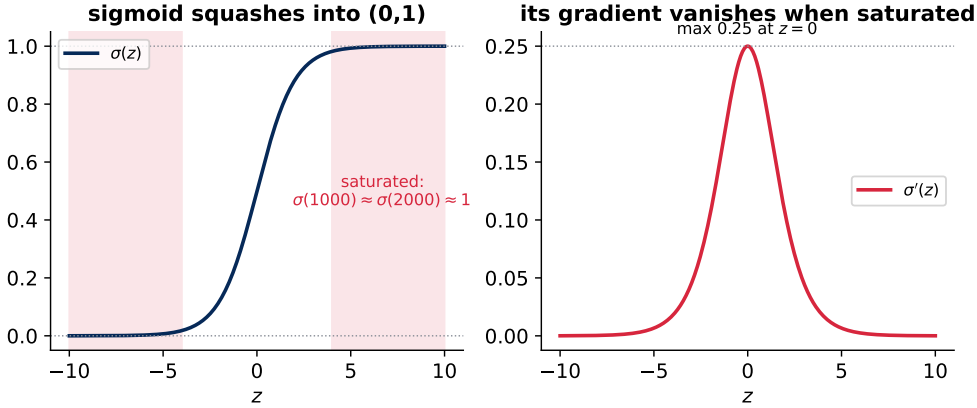
C. Because ReLU is differentiable everywhere, while sigmoid is not.

× Backwards: sigmoid is smooth *everywhere*; ReLU is the one with a kink at $z = 0$.

D. Because ReLU guarantees the network will not overfit.

× No activation function can *guarantee* no overfitting.

Demo: sigmoid saturation and its vanishing gradient



Q25 — Zero training loss

When training the logistic regression model, we use gradient descent to minimize the loss function. What happens if the loss function became zero?

A. This means the model's prediction will always be zero.

B. This means our code is wrong, because it is impossible.

C. This means the model now fit the training dataset really well.

D. This means the model now fit the test dataset really well.

Q25 — Zero training loss — Answer

A. This means the model's prediction will always be zero.

× Zero *loss* means predictions match labels — not that predictions are zero.

Q25 — Zero training loss — Answer

A. This means the model's prediction will always be zero.

× Zero *loss* means predictions match labels — not that predictions are zero.

B. This means our code is wrong, because it is impossible.

× Reaching (numerically) zero is possible — and often a warning sign, not a bug.

Q25 — Zero training loss — Answer

A. This means the model's prediction will always be zero.

× Zero *loss* means predictions match labels — not that predictions are zero.

B. This means our code is wrong, because it is impossible.

× Reaching (numerically) zero is possible — and often a warning sign, not a bug.

C. This means the model now fit the training dataset really well.

✓ Perfect fit on the *training* data. It promises nothing beyond it — and may signal overfitting.

Q25 — Zero training loss — Answer

A. This means the model's prediction will always be zero.

× Zero *loss* means predictions match labels — not that predictions are zero.

B. This means our code is wrong, because it is impossible.

× Reaching (numerically) zero is possible — and often a warning sign, not a bug.

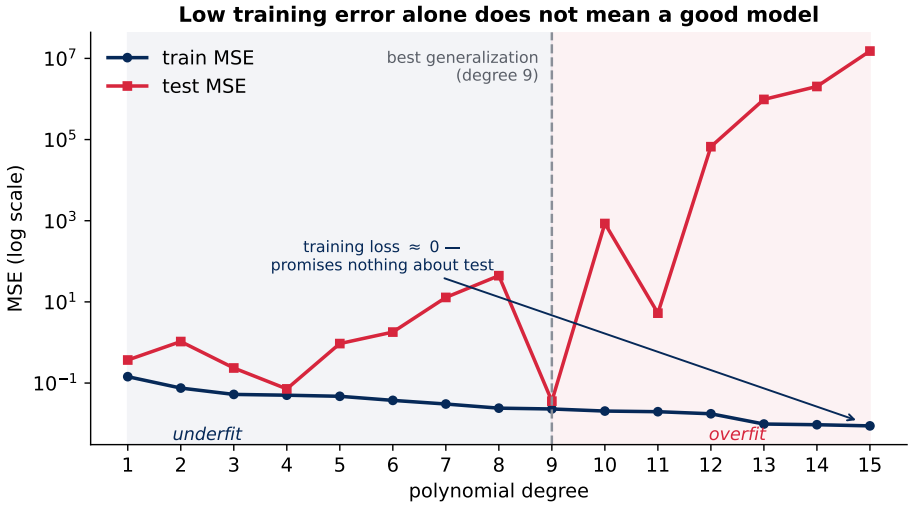
C. This means the model now fit the training dataset really well.

✓ Perfect fit on the *training* data. It promises nothing beyond it — and may signal overfitting.

D. This means the model now fit the test dataset really well.

× The training loss never saw the test set — it cannot certify test performance.

Demo: zero training loss $\neq$ good model



Q26 — Logistic regression output

The output of logistic regression is:

A. A continuous value

B. A probability score

C. A categorical label

D. A binary value

Q26 — Logistic regression output — Answer

Review

Because the model outputs a probability rather than a hard label, the classification threshold is yours to tune — which is what makes logistic regression adaptable to imbalanced problems.

Q26 — Logistic regression output — Answer

Review

Because the model outputs a probability rather than a hard label, the classification threshold is yours to tune — which is what makes logistic regression adaptable to imbalanced problems.

A. A continuous value

× "A continuous value" describes *linear* regression's unbounded output — logistic is bounded in $(0, 1)$.

Q26 — Logistic regression output — Answer

Review

Because the model outputs a probability rather than a hard label, the classification threshold is yours to tune — which is what makes logistic regression adaptable to imbalanced problems.

A. A continuous value

× "A continuous value" describes *linear* regression's unbounded output — logistic is bounded in $(0, 1)$.

B. A probability score

✓ Sigmoid$(w^\top x + b)$ gives $P(y{=}1 \mid x)$ — a probability score.

Q26 — Logistic regression output — Answer

Review

Because the model outputs a probability rather than a hard label, the classification threshold is yours to tune — which is what makes logistic regression adaptable to imbalanced problems.

A. A continuous value

× "A continuous value" describes *linear* regression's unbounded output — logistic is bounded in $(0, 1)$.

B. A probability score

✓ Sigmoid$(w^\top x + b)$ gives $P(y{=}1 \mid x)$ — a probability score.

C. A categorical label

× The categorical label appears only *after you* threshold the probability.

Q26 — Logistic regression output — Answer

Review

Because the model outputs a probability rather than a hard label, the classification threshold is yours to tune — which is what makes logistic regression adaptable to imbalanced problems.

A. A continuous value

× "A continuous value" describes *linear* regression's unbounded output — logistic is bounded in $(0, 1)$.

B. A probability score

✓ Sigmoid$(w^\top x + b)$ gives $P(y{=}1 \mid x)$ — a probability score.

C. A categorical label

× The categorical label appears only *after you* threshold the probability.

D. A binary value

× Same — the 0/1 comes from applying the 0.5 cutoff, not from the model itself.

Q27 — Decision tree splits

How is the splitting attribute chosen in a decision tree?

A. Randomly selecting an attribute from the dataset.

B. Choosing the attribute that has the highest information gain or the lowest Gini impurity.

C. Using gradient descent to find the best attribute.

D. Selecting the attribute that has the highest correlation with the target variable.

Q27 — Decision tree splits — Answer

A. Randomly selecting an attribute from the dataset.

× Splits are chosen *greedily by purity*, not at random. (Random forests subsample *candidate* features, but still pick the best split among them.)

Q27 — Decision tree splits — Answer

A. Randomly selecting an attribute from the dataset.

× Splits are chosen *greedily by purity*, not at random. (Random forests subsample *candidate* features, but still pick the best split among them.)

B. Choosing the attribute that has the highest information gain or the lowest Gini impurity.

✓ Try every attribute and threshold, keep the split with highest information gain / lowest Gini.

Q27 — Decision tree splits — Answer

A. Randomly selecting an attribute from the dataset.

× Splits are chosen *greedily by purity*, not at random. (Random forests subsample *candidate* features, but still pick the best split among them.)

B. Choosing the attribute that has the highest information gain or the lowest Gini impurity.

✓ Try every attribute and threshold, keep the split with highest information gain / lowest Gini.

C. Using gradient descent to find the best attribute.

× Splitting is not differentiable — gradient descent applies to logistic regression and neural nets instead.

Q27 — Decision tree splits — Answer

A. Randomly selecting an attribute from the dataset.

× Splits are chosen *greedily by purity*, not at random. (Random forests subsample *candidate* features, but still pick the best split among them.)

B. Choosing the attribute that has the highest information gain or the lowest Gini impurity.

✓ Try every attribute and threshold, keep the split with highest information gain / lowest Gini.

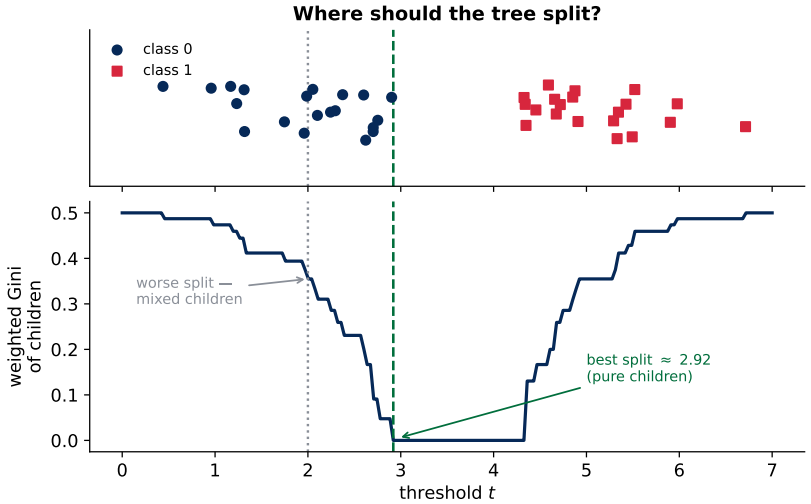
C. Using gradient descent to find the best attribute.

× Splitting is not differentiable — gradient descent applies to logistic regression and neural nets instead.

D. Selecting the attribute that has the highest correlation with the target variable.

× Correlation only captures *linear* relations — trees split on impurity reduction.

Demo: sweeping thresholds to find the best split



Q28 — Cost function

What is the purpose of a cost function?

A. The cost function is used to calculate the total number of data points in the training set.

B. The cost function is a measure of how good or bad the decision tree fits the training data.

C. The cost function is used to evaluate the complexity of the decision tree.

D. The cost function is only applicable to unsupervised learning algorithms, not decision trees.

Q28 — Cost function — Answer

A. The cost function is used to calculate the total number of data points in the training set.

× Dataset size is `len(data)` — no function needed.

Q28 — Cost function — Answer

A. The cost function is used to calculate the total number of data points in the training set.

× Dataset size is `len(data)` — no function needed.

B. The cost function is a measure of how good or bad the decision tree fits the training data.

✓ It turns "how wrong is the model" into one number the algorithm can minimize.

Q28 — Cost function — Answer

A. The cost function is used to calculate the total number of data points in the training set.

× Dataset size is `len(data)` — no function needed.

B. The cost function is a measure of how good or bad the decision tree fits the training data.

✓ It turns "how wrong is the model" into one number the algorithm can minimize.

C. The cost function is used to evaluate the complexity of the decision tree.

× Complexity is measured by depth/leaf count (or a regularization *term added to* the cost).

Q28 — Cost function — Answer

A. The cost function is used to calculate the total number of data points in the training set.

× Dataset size is `len(data)` — no function needed.

B. The cost function is a measure of how good or bad the decision tree fits the training data.

✓ It turns "how wrong is the model" into one number the algorithm can minimize.

C. The cost function is used to evaluate the complexity of the decision tree.

× Complexity is measured by depth/leaf count (or a regularization *term added to* the cost).

D. The cost function is only applicable to unsupervised learning algorithms, not decision trees.

× Cost functions are everywhere in *supervised* learning — backwards.

Q29 — Ensemble learning

What is ensemble learning?

A. A single machine learning model

B. A group of unrelated models

C. A technique that combines predictions from multiple models

D. A method to visualize data

Q29 — Ensemble learning — Answer

Review

The committee beats its average member — the principle behind random forests, bagging, and boosting.

Q29 — Ensemble learning — Answer

Review

The committee beats its average member — the principle behind random forests, bagging, and boosting.

A. A single machine learning model

× An ensemble is by definition *more than one* model.

Q29 — Ensemble learning — Answer

Review

The committee beats its average member — the principle behind random forests, bagging, and boosting.

A. A single machine learning model

× An ensemble is by definition *more than one* model.

B. A group of unrelated models

× "Unrelated" misses the point — the models' predictions must be *combined*.

Q29 — Ensemble learning — Answer

Review

The committee beats its average member — the principle behind random forests, bagging, and boosting.

A. A single machine learning model

× An ensemble is by definition *more than one* model.

B. A group of unrelated models

× "Unrelated" misses the point — the models' predictions must be *combined*.

C. A technique that combines predictions from multiple models

✓ Vote or average across models: errors that do not repeat across members cancel out.

Q29 — Ensemble learning — Answer

Review

The committee beats its average member — the principle behind random forests, bagging, and boosting.

A. A single machine learning model

× An ensemble is by definition *more than one* model.

B. A group of unrelated models

× "Unrelated" misses the point — the models' predictions must be *combined*.

C. A technique that combines predictions from multiple models

✓ Vote or average across models: errors that do not repeat across members cancel out.

D. A method to visualize data

× Visualization is unrelated.

Q30 — Bootstrapping

In Random Forest, what is bootstrapping used for?

A. Enhancing model interpretability

B. Preventing overfitting and speed up training

C. Generating different random subsets of data for training

D. Improving the convergence of the algorithm

Q30 — Bootstrapping — Answer

A. Enhancing model interpretability

× 100 resampled trees are *less* interpretable than one tree, not more.

Q30 — Bootstrapping — Answer

A. Enhancing model interpretability

× 100 resampled trees are *less* interpretable than one tree, not more.

B. Preventing overfitting and speed up training

× Half-true at best: sampling does not speed up training, and the overfitting reduction comes from *averaging*, not the sampling itself.

Q30 — Bootstrapping — Answer

A. Enhancing model interpretability

× 100 resampled trees are *less* interpretable than one tree, not more.

B. Preventing overfitting and speed up training

× Half-true at best: sampling does not speed up training, and the overfitting reduction comes from *averaging*, not the sampling itself.

C. Generating different random subsets of data for training

✓ Each tree gets a random sample of rows *with replacement* ($\approx 2/3$ of distinct rows) — different data $\Rightarrow$ different trees $\Rightarrow$ a stable averaged vote.

Q30 — Bootstrapping — Answer

A. Enhancing model interpretability

× 100 resampled trees are *less* interpretable than one tree, not more.

B. Preventing overfitting and speed up training

× Half-true at best: sampling does not speed up training, and the overfitting reduction comes from *averaging*, not the sampling itself.

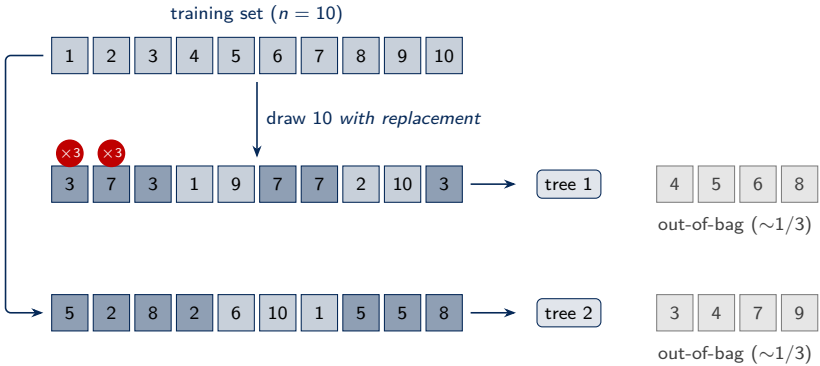
C. Generating different random subsets of data for training

✓ Each tree gets a random sample of rows *with replacement* ($\approx 2/3$ of distinct rows) — different data $\Rightarrow$ different trees $\Rightarrow$ a stable averaged vote.

D. Improving the convergence of the algorithm

× "Convergence" is an optimizer concept — trees are built greedily, not iterated to convergence.

Demo: bootstrap sampling with replacement



Q31 — Activation functions

Why are activation functions important in MLPs?

A. They allow for the introduction of linear transformations.

B. They determine the number of layers in the network.

C. They introduce non-linearity, enabling the model to learn complex patterns.

D. They reduce the computational complexity of the network.

Q31 — Activation functions — Answer

A. They allow for the introduction of linear transformations.

× Layers are already linear — activations exist to add the *opposite*.

Q31 — Activation functions — Answer

A. They allow for the introduction of linear transformations.

× Layers are already linear — activations exist to add the *opposite*.

B. They determine the number of layers in the network.

× You choose the number of layers; activations play no part in it.

Q31 — Activation functions — Answer

A. They allow for the introduction of linear transformations.

× Layers are already linear — activations exist to add the *opposite*.

B. They determine the number of layers in the network.

× You choose the number of layers; activations play no part in it.

C. They introduce non-linearity, enabling the model to learn complex patterns.

✓ Without non-linearity, any stack of linear layers collapses into a single matrix multiply.

Q31 — Activation functions — Answer

A. They allow for the introduction of linear transformations.

× Layers are already linear — activations exist to add the *opposite*.

B. They determine the number of layers in the network.

× You choose the number of layers; activations play no part in it.

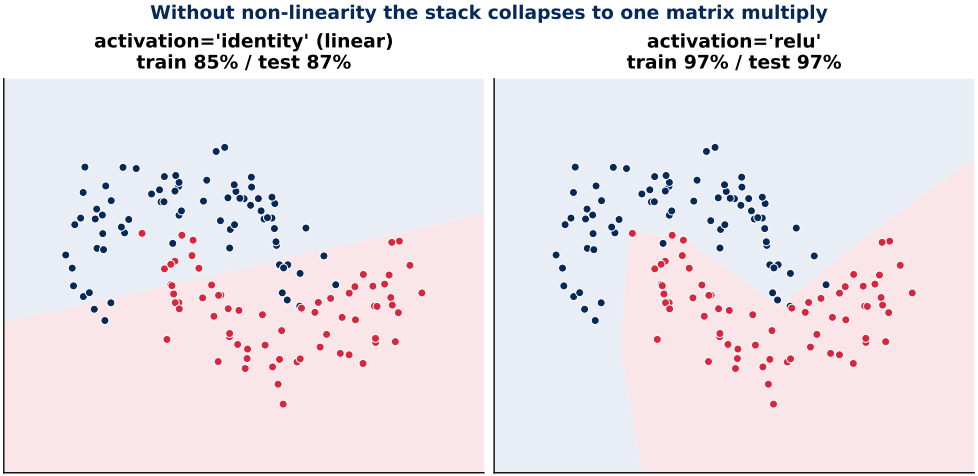
C. They introduce non-linearity, enabling the model to learn complex patterns.

✓ Without non-linearity, any stack of linear layers collapses into a single matrix multiply.

D. They reduce the computational complexity of the network.

× Activations *add* a little computation, they do not reduce it.

Demo: same layers, with and without non-linearity



Q32 — Sigmoid properties

Which of the following is **NOT** a property of the sigmoid function?

A. Differentiable for all real numbers

B. Output range between 0 and 1

C. The result of sigmoid(2000) is twice as large as sigmoid(1000)

D. Continuous for all real numbers

Q32 — Sigmoid properties — Answer

Review

Sigmoid's flat, saturated tails are the same fact behind the vanishing-gradient problem from Q24.

Q32 — Sigmoid properties — Answer

Review

Sigmoid's flat, saturated tails are the same fact behind the vanishing-gradient problem from Q24.

A. Differentiable for all real numbers

× IS a property — sigmoid is differentiable everywhere.

Q32 — Sigmoid properties — Answer

Review

Sigmoid's flat, saturated tails are the same fact behind the vanishing-gradient problem from Q24.

A. Differentiable for all real numbers

× IS a property — sigmoid is differentiable everywhere.

B. Output range between 0 and 1

× IS a property — outputs squash into $(0, 1)$.

Q32 — Sigmoid properties — Answer

Review

Sigmoid's flat, saturated tails are the same fact behind the vanishing-gradient problem from Q24.

A. Differentiable for all real numbers

× IS a property — sigmoid is differentiable everywhere.

B. Output range between 0 and 1

× IS a property — outputs squash into $(0, 1)$.

C. The result of sigmoid(2000) is twice as large as sigmoid(1000)

✓ By $z \approx 6$ sigmoid has saturated at ≈ 1: $\sigma(2000)$ and $\sigma(1000)$ are both essentially 1, not doubled. Sigmoid is not linear.

Q32 — Sigmoid properties — Answer

Review

Sigmoid's flat, saturated tails are the same fact behind the vanishing-gradient problem from Q24.

A. Differentiable for all real numbers

× IS a property — sigmoid is differentiable everywhere.

B. Output range between 0 and 1

× IS a property — outputs squash into $(0, 1)$.

C. The result of sigmoid(2000) is twice as large as sigmoid(1000)

✓ By $z \approx 6$ sigmoid has saturated at ≈ 1: $\sigma(2000)$ and $\sigma(1000)$ are both essentially 1, not doubled. Sigmoid is not linear.

D. Continuous for all real numbers

× IS a property — sigmoid is continuous everywhere.

Q33 — Backprop chain rule

A single sigmoid neuron takes input $x = 1$, with weight $w = 1$ and bias $b = -1$, so that $z = wx + b$. The output is $a = \sigma(z)$, and the cost is $C = \frac{1}{2}(a - y)^2$ with target $y = 1$.

You are given the element derivatives:
$\partial C/\partial a = a - y$
$\partial a/\partial z = \sigma'(z) = \sigma(z)\,(1 - \sigma(z))$
$\partial z/\partial w = x$
Using the chain rule, what is $\partial C/\partial w$?

A. -0.5

B. -0.125

C. $+0.125$

D. -0.0625

Q33 — Backprop chain rule — Answer

Review

Each distractor is a classic chain-rule mistake: -0.5 stops at $\partial C/\partial a$; $+0.125$ writes $(y-a)$ instead of $(a-y)$; -0.0625 uses a instead of x for $\partial z/\partial w$.

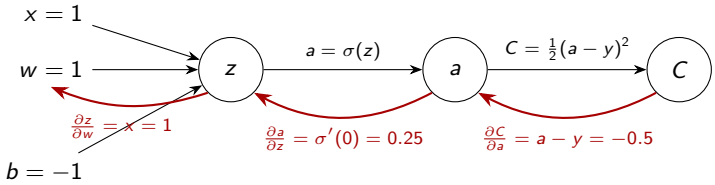
Q33 — Backprop chain rule — Answer

Review

Each distractor is a classic chain-rule mistake: -0.5 stops at $\partial C/\partial a$; $+0.125$ writes $(y-a)$ instead of $(a-y)$; -0.0625 uses a instead of x for $\partial z/\partial w$.

✓ **Answer: B.** Forward: $z = 1 \cdot 1 - 1 = 0$, so $a = \sigma(0) = 0.5$ and $\sigma'(0) = 0.25$. Chain rule: $\frac{\partial C}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w} = (-0.5)(0.25)(1) = \mathbf{-0.125}$.

Demo: the chain rule on one neuron



$$\frac{\partial C}{\partial w} = (-0.5) \times (0.25) \times (1) = \mathbf{-0.125}$$

Q34 — What is an MLP

Which of the following best describes a Multi-layer Perceptron (MLP)?

A. A single-layer feedforward neural network.

B. A neural network that primarily uses convolutional layers.

C. A type of recurrent neural network optimized for time-series data.

D. A feedforward neural network with one or more hidden layers.

Q34 — What is an MLP — Answer

A. A single-layer feedforward neural network.

× A single-layer feedforward network is the classic *perceptron* — the opposite of *multi*-layer.

Q34 — What is an MLP — Answer

A. A single-layer feedforward neural network.

× A single-layer feedforward network is the classic *perceptron* — the opposite of *multi*-layer.

B. A neural network that primarily uses convolutional layers.

× Convolution-first networks are *CNNs*.

Q34 — What is an MLP — Answer

A. A single-layer feedforward neural network.

× A single-layer feedforward network is the classic *perceptron* — the opposite of *multi*-layer.

B. A neural network that primarily uses convolutional layers.

× Convolution-first networks are *CNNs*.

C. A type of recurrent neural network optimized for time-series data.

× Recurrence for time series is an *RNN*.

Q34 — What is an MLP — Answer

A. A single-layer feedforward neural network.

× A single-layer feedforward network is the classic *perceptron* — the opposite of *multi*-layer.

B. A neural network that primarily uses convolutional layers.

× Convolution-first networks are *CNNs*.

C. A type of recurrent neural network optimized for time-series data.

× Recurrence for time series is an *RNN*.

D. A feedforward neural network with one or more hidden layers.

✓ Input → one or more hidden layers → output, strictly feedforward: an MLP.

Q35 — Why convolution

Which of the following is a primary reason for using convolutional layers in a neural network when processing image data?

A. They increase the number of parameters in the network.

B. They allow the network to be fully connected.

C. They are designed to detect local patterns, like edges and textures.

D. They ensure that the network only processes black and white images.

Q35 — Why convolution — Answer

A. They increase the number of parameters in the network.

× Backwards — weight sharing means convolution needs *far fewer* parameters than dense layers on pixels.

Q35 — Why convolution — Answer

A. They increase the number of parameters in the network.

× Backwards — weight sharing means convolution needs *far fewer* parameters than dense layers on pixels.

B. They allow the network to be fully connected.

× Fully-connected is what convolution *replaces* for images.

Q35 — Why convolution — Answer

A. They increase the number of parameters in the network.

× Backwards — weight sharing means convolution needs *far fewer* parameters than dense layers on pixels.

B. They allow the network to be fully connected.

× Fully-connected is what convolution *replaces* for images.

C. They are designed to detect local patterns, like edges and textures.

✓ A small filter slides across the image, firing on the same local pattern (edge, texture) wherever it appears.

Q35 — Why convolution — Answer

A. They increase the number of parameters in the network.

× Backwards — weight sharing means convolution needs *far fewer* parameters than dense layers on pixels.

B. They allow the network to be fully connected.

× Fully-connected is what convolution *replaces* for images.

C. They are designed to detect local patterns, like edges and textures.

✓ A small filter slides across the image, firing on the same local pattern (edge, texture) wherever it appears.

D. They ensure that the network only processes black and white images.

× CNNs handle color naturally — channels.

Demo: one filter detecting edges everywhere

- A filter *is* just a small table of weights — here 3×3, hand-set to $+1/0/-1$ as a vertical-edge detector.
- At each position: multiply the 9 pixels under the window by the 9 weights and sum = (left column) − (right column). Uniform region ⇒ cancels to 0; brightness change ⇒ large response.
- In a real CNN these 9 numbers are **learned** — `Conv2D(32, (3,3))` learns 32 such tables, each specializing in a different local pattern.

Q36 — Pooling

In a CNN, what does the term ‘pooling’ generally refer to?

A. Increasing the depth of the network by adding more convolutional layers.

B. The process of periodically saving the network’s weights to avoid data loss.

C. Reducing the spatial dimensions of the data by taking the average or maximum value from a group of neighboring pixels.

D. Applying a filter to increase the sharpness of the image before processing.

Q36 — Pooling — Answer

Review

The point of pooling is *translation invariance*: the network should care that a feature is roughly there, not at which exact pixel.

Q36 — Pooling — Answer

Review

The point of pooling is *translation invariance*: the network should care that a feature is roughly there, not at which exact pixel.

A. Increasing the depth of the network by adding more convolutional layers.

× Depth comes from stacking *conv* layers, not from pooling.

Q36 — Pooling — Answer

Review

The point of pooling is *translation invariance*: the network should care that a feature is roughly there, not at which exact pixel.

A. Increasing the depth of the network by adding more convolutional layers.

× Depth comes from stacking *conv* layers, not from pooling.

B. The process of periodically saving the network's weights to avoid data loss.

× Saving weights is *checkpointing* — a training-infrastructure concern.

Q36 — Pooling — Answer

Review

The point of pooling is *translation invariance*: the network should care that a feature is roughly there, not at which exact pixel.

A. Increasing the depth of the network by adding more convolutional layers.

× Depth comes from stacking *conv* layers, not from pooling.

B. The process of periodically saving the network's weights to avoid data loss.

× Saving weights is *checkpointing* — a training-infrastructure concern.

C. Reducing the spatial dimensions of the data by taking the average or maximum value from a group of neighboring pixels.

✓ Summarize each neighbourhood by its max/average: smaller feature maps, slight translation invariance.

Q36 — Pooling — Answer

Review

The point of pooling is *translation invariance*: the network should care that a feature is roughly there, not at which exact pixel.

A. Increasing the depth of the network by adding more convolutional layers.

× Depth comes from stacking *conv* layers, not from pooling.

B. The process of periodically saving the network's weights to avoid data loss.

× Saving weights is *checkpointing* — a training-infrastructure concern.

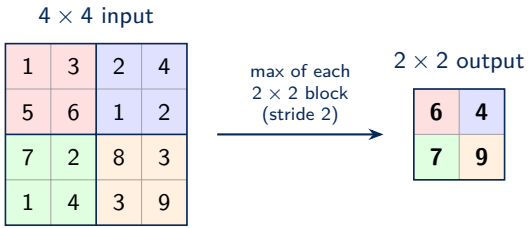
C. Reducing the spatial dimensions of the data by taking the average or maximum value from a group of neighboring pixels.

✓ Summarize each neighbourhood by its max/average: smaller feature maps, slight translation invariance.

D. Applying a filter to increase the sharpness of the image before processing.

× Sharpening is an image filter, not a CNN operation.

Demo: max pooling on a 4×4 grid



average pooling: 3.75, 2.25, 3.5, 5.75 — same shrink, softer summary

Q37 — CNN activations

Which of the following is **NOT** a common activation function used in CNNs?

A. ReLU (Rectified Linear Unit)

B. Tanh

C. Leaky ReLU

D. Softmax

Q37 — CNN activations — Answer

A. ReLU (Rectified Linear Unit)

× ReLU is THE standard hidden activation in CNNs.

Q37 — CNN activations — Answer

A. ReLU (Rectified Linear Unit)

× ReLU is THE standard hidden activation in CNNs.

B. Tanh

× Tanh is used too — this course's from-scratch CNN uses it.

Q37 — CNN activations — Answer

A. ReLU (Rectified Linear Unit)

× ReLU is THE standard hidden activation in CNNs.

B. Tanh

× Tanh is used too — this course's from-scratch CNN uses it.

C. Leaky ReLU

× Leaky ReLU is a common ReLU variant (fixes dead neurons).

Q37 — CNN activations — Answer

A. ReLU (Rectified Linear Unit)

× ReLU is THE standard hidden activation in CNNs.

B. Tanh

× Tanh is used too — this course's from-scratch CNN uses it.

C. Leaky ReLU

× Leaky ReLU is a common ReLU variant (fixes dead neurons).

D. Softmax

✓ Softmax normalizes a whole vector into class probabilities — it lives at the *output* layer, not between conv layers.

Q38 — Stride

In the context of CNNs, what does a ‘stride’ refer to?

A. The depth of the convolutional filter.

B. The size of the pooling window.

C. The number of layers in the network.

D. The number of pixels the convolutional filter moves after processing a section of the input.

Q38 — Stride — Answer

A. The depth of the convolutional filter.

× Filter *depth* equals input channels; the number of filters sets output channels.

Q38 — Stride — Answer

A. The depth of the convolutional filter.

× Filter *depth* equals input channels; the number of filters sets output channels.

B. The size of the pooling window.

× The pooling window belongs to *pooling* layers.

Q38 — Stride — Answer

A. The depth of the convolutional filter.

× Filter *depth* equals input channels; the number of filters sets output channels.

B. The size of the pooling window.

× The pooling window belongs to *pooling* layers.

C. The number of layers in the network.

× Layer count is the architecture, not a per-layer setting.

Q38 — Stride — Answer

A. The depth of the convolutional filter.

× Filter *depth* equals input channels; the number of filters sets output channels.

B. The size of the pooling window.

× The pooling window belongs to *pooling* layers.

C. The number of layers in the network.

× Layer count is the architecture, not a per-layer setting.

D. The number of pixels the convolutional filter moves after processing a section of the input.

✓ Stride = how many pixels the filter steps between applications; stride 2 halves the output size.

Q39 — RNN hidden layer

In a Recurrent Neural Network, what is the primary purpose of the hidden layer?

A. To process the input data and produce the final output.

B. To store sequential information for memory

C. To apply activation functions to the input features.

D. To directly connect with the output layer.

Q39 — RNN hidden layer — Answer

Review

The hidden state is updated each step from the current input *and* the previous hidden state — $h_t = f(x_t, h_{t-1})$ — which is how earlier context reaches later steps.

Q39 — RNN hidden layer — Answer

Review

The hidden state is updated each step from the current input *and* the previous hidden state — $h_t = f(x_t, h_{t-1})$ — which is how earlier context reaches later steps.

A. To process the input data and produce the final output.

× Producing the final output is the *output* layer's job.

Q39 — RNN hidden layer — Answer

Review

The hidden state is updated each step from the current input *and* the previous hidden state — $h_t = f(x_t, h_{t-1})$ — which is how earlier context reaches later steps.

A. To process the input data and produce the final output.

× Producing the final output is the *output* layer's job.

B. To store sequential information for memory

✓ h_t carries information from earlier steps forward — the network's memory, and the defining feature of recurrence.

Q39 — RNN hidden layer — Answer

Review

The hidden state is updated each step from the current input *and* the previous hidden state — $h_t = f(x_t, h_{t-1})$ — which is how earlier context reaches later steps.

A. To process the input data and produce the final output.

× Producing the final output is the *output* layer's job.

B. To store sequential information for memory

✓ h_t carries information from earlier steps forward — the network's memory, and the defining feature of recurrence.

C. To apply activation functions to the input features.

× Activations apply in every layer — not the hidden layer's defining role.

Q39 — RNN hidden layer — Answer

Review

The hidden state is updated each step from the current input *and* the previous hidden state — $h_t = f(x_t, h_{t-1})$ — which is how earlier context reaches later steps.

A. To process the input data and produce the final output.

× Producing the final output is the *output* layer's job.

B. To store sequential information for memory

✓ h_t carries information from earlier steps forward — the network's memory, and the defining feature of recurrence.

C. To apply activation functions to the input features.

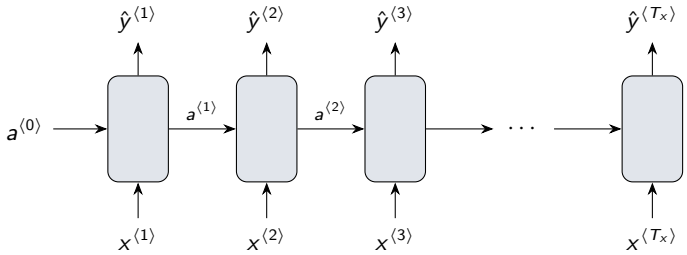
× Activations apply in every layer — not the hidden layer's defining role.

D. To directly connect with the output layer.

× Skipping straight to output would discard the recurrence that makes it an RNN.

Q40 — RNN architecture use case

Suppose you have the following RNN architecture. In what cases would you use it?



A. Action Recognition in a video

B. Sentiment Classification

C. Image Recognition

D. Named Entity Recognition

Q40 — RNN architecture use case — Answer

A. Action Recognition in a video

× Action recognition maps a whole clip to ONE label — many-to-one.

Q40 — RNN architecture use case — Answer

A. Action Recognition in a video

× Action recognition maps a whole clip to ONE label — many-to-one.

B. Sentiment Classification

× Sentiment is many-to-one as well.

Q40 — RNN architecture use case — Answer

A. Action Recognition in a video

× Action recognition maps a whole clip to ONE label — many-to-one.

B. Sentiment Classification

× Sentiment is many-to-one as well.

C. Image Recognition

× Image recognition is not sequential at all — use a CNN.

Q40 — RNN architecture use case — Answer

A. Action Recognition in a video

× Action recognition maps a whole clip to ONE label — many-to-one.

B. Sentiment Classification

× Sentiment is many-to-one as well.

C. Image Recognition

× Image recognition is not sequential at all — use a CNN.

D. Named Entity Recognition

✓ One $\hat{y}$ per input token, $T_x = T_y$: exactly the diagram. One tag per word = NER.

Q41 — RNN advantage

What is the main advantage of using a Recurrent Neural Network (RNN) compared to a traditional feedforward neural network?

A. RNNs can predict on input sequences of any length

B. RNNs have more layers, leading to better performance.

C. RNNs are faster in training due to parallelization.

D. RNNs require less memory for computation.

Q41 — RNN advantage — Answer

A. RNNs can predict on input sequences of any length

✓ The same cell (same weights) is reused each step — any sequence length, fixed parameter count.

Q41 — RNN advantage — Answer

A. RNNs can predict on input sequences of any length

✓ The same cell (same weights) is reused each step — any sequence length, fixed parameter count.

B. RNNs have more layers, leading to better performance.

× Depth is orthogonal — both families can be deep.

Q41 — RNN advantage — Answer

A. RNNs can predict on input sequences of any length

✓ The same cell (same weights) is reused each step — any sequence length, fixed parameter count.

B. RNNs have more layers, leading to better performance.

× Depth is orthogonal — both families can be deep.

C. RNNs are faster in training due to parallelization.

× Exactly backwards: RNNs cannot parallelize across time.

Q41 — RNN advantage — Answer

A. RNNs can predict on input sequences of any length

✓ The same cell (same weights) is reused each step — any sequence length, fixed parameter count.

B. RNNs have more layers, leading to better performance.

× Depth is orthogonal — both families can be deep.

C. RNNs are faster in training due to parallelization.

× Exactly backwards: RNNs cannot parallelize across time.

D. RNNs require less memory for computation.

× Memory footprint is not the distinguishing advantage.

Q42 — Many-to-one RNN

To which of the following you would use many-to-one architecture of RNN?

A. Sentiment Classification

B. Question Answering

C. Named Entity Recognition

D. Music Generation

Q42 — Many-to-one RNN — Answer

A. Sentiment Classification

✓ Whole review in, one sentiment label out — the textbook many-to-one task.

Q42 — Many-to-one RNN — Answer

A. Sentiment Classification

✓ Whole review in, one sentiment label out — the textbook many-to-one task.

B. Question Answering

× Question answering *produces a sequence* — not a single output.

Q42 — Many-to-one RNN — Answer

A. Sentiment Classification

✓ Whole review in, one sentiment label out — the textbook many-to-one task.

B. Question Answering

× Question answering *produces a sequence* — not a single output.

C. Named Entity Recognition

× NER is many-to-many, one tag per token (Q40).

Q42 — Many-to-one RNN — Answer

A. Sentiment Classification

✓ Whole review in, one sentiment label out — the textbook many-to-one task.

B. Question Answering

× Question answering *produces a sequence* — not a single output.

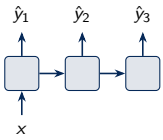
C. Named Entity Recognition

× NER is many-to-many, one tag per token (Q40).

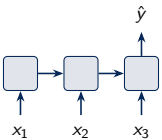
D. Music Generation

× Music generation is one-to-many: a seed in, a sequence out.

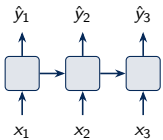
Demo: the four RNN shapes



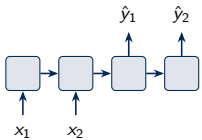
one-to-many
music generation



many-to-one
sentiment classification



many-to-many ($T_x = T_y$)
NER (Q40)



encoder–decoder
QA / translation

Q43 — NaN weights

You are training an RNN based architecture and realized during the training that the weights have a value of NaN. Which of these is a possible reason for the same?

A. Because the RNN cell uses sigmoid activation function.

B. Because it uses ReLU activation function.

C. Vanishing gradient problem

D. Exploding gradient problem

Q43 — NaN weights — Answer

Review

Backprop *through time* multiplies gradient factors at every timestep, so over long sequences gradients grow or shrink *exponentially*. Both failure modes have names — exploding and vanishing — with opposite symptoms, and RNN design choices (bounded activations like tanh, gradient clipping) exist to manage them.

Q43 — NaN weights — Answer

Review

Backprop *through time* multiplies gradient factors at every timestep, so over long sequences gradients grow or shrink *exponentially*. Both failure modes have names — exploding and vanishing — with opposite symptoms, and RNN design choices (bounded activations like tanh, gradient clipping) exist to manage them.

A. Because the RNN cell uses sigmoid activation function.

× Sigmoid's bounded output is linked to *vanishing* gradients — the opposite failure.

Q43 — NaN weights — Answer

Review

Backprop *through time* multiplies gradient factors at every timestep, so over long sequences gradients grow or shrink *exponentially*. Both failure modes have names — exploding and vanishing — with opposite symptoms, and RNN design choices (bounded activations like tanh, gradient clipping) exist to manage them.

A. Because the RNN cell uses sigmoid activation function.

× Sigmoid's bounded output is linked to *vanishing* gradients — the opposite failure.

B. Because it uses ReLU activation function.

× ReLU's unboundedness can feed the growth, but the failure mode that produces NaN has a name — option D.

Q43 — NaN weights — Answer

Review

Backprop *through time* multiplies gradient factors at every timestep, so over long sequences gradients grow or shrink *exponentially*. Both failure modes have names — exploding and vanishing — with opposite symptoms, and RNN design choices (bounded activations like tanh, gradient clipping) exist to manage them.

A. Because the RNN cell uses sigmoid activation function.

× Sigmoid's bounded output is linked to *vanishing* gradients — the opposite failure.

B. Because it uses ReLU activation function.

× ReLU's unboundedness can feed the growth, but the failure mode that produces NaN has a name — option D.

C. Vanishing gradient problem

× Vanishing gradients stall learning *silently*; the numbers stay finite, never NaN.

Q43 — NaN weights — Answer

Review

Backprop *through time* multiplies gradient factors at every timestep, so over long sequences gradients grow or shrink *exponentially*. Both failure modes have names — exploding and vanishing — with opposite symptoms, and RNN design choices (bounded activations like tanh, gradient clipping) exist to manage them.

A. Because the RNN cell uses sigmoid activation function.

× Sigmoid's bounded output is linked to *vanishing* gradients — the opposite failure.

B. Because it uses ReLU activation function.

× ReLU's unboundedness can feed the growth, but the failure mode that produces NaN has a name — option D.

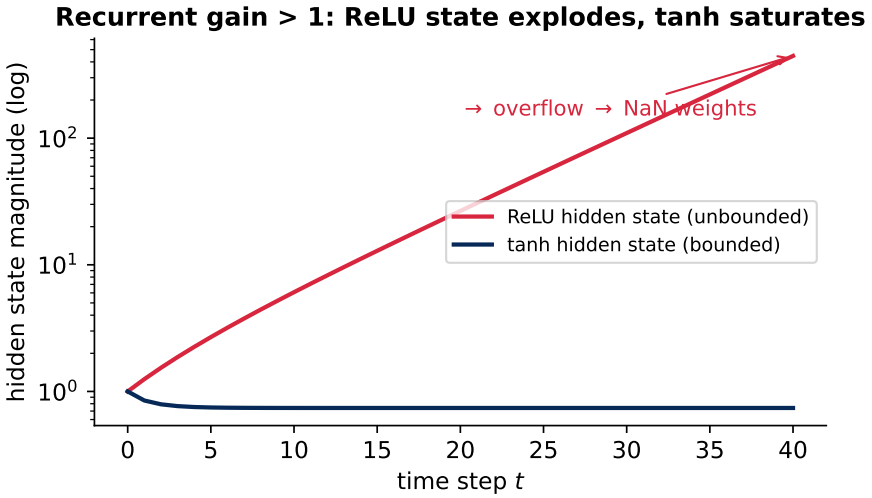
C. Vanishing gradient problem

× Vanishing gradients stall learning *silently*; the numbers stay finite, never NaN.

D. Exploding gradient problem

✓ Backprop-through-time multiplies factors > 1 exponentially $\rightarrow$ overflow to `inf` $\rightarrow$ `inf`$-$`inf` $=$ NaN.

Demo: unbounded hidden state → exploding values



Q44 — pandas .loc

Which of the following loc commands correctly selects the row with the name ‘Charlie’ and all columns?

	Name	Age	Score
0	Alice	25	90
1	Bob	18	82
2	Charlie	12	70
3	David	30	95

A. `df.loc[2]`

B. `df.loc['Charlie']`

C. `df.loc[df['Name'] == 'Charlie']`

D. `df.loc[2, :]`

Q44 — pandas .loc — Answer

Review

`.loc` selects rows by *label* — an index label, a label-plus-columns pair, or a boolean mask — so several idioms can legitimately reach the same row. **Grading note:** three of the four options select Charlie's row (verified by running them); grade all three as correct. For a future form, flip the stem to "which does NOT select Charlie's row?"

Q44 — pandas .loc — Answer

Review

`.loc` selects rows by *label* — an index label, a label-plus-columns pair, or a boolean mask — so several idioms can legitimately reach the same row. **Grading note:** three of the four options select Charlie's row (verified by running them); grade all three as correct. For a future form, flip the stem to "which does NOT select Charlie's row?"

A. `df.loc[2]`

✓ *Also works*: returns the same row as a Series — accepted for credit.

Q44 — pandas .loc — Answer

Review

`.loc` selects rows by *label* — an index label, a label-plus-columns pair, or a boolean mask — so several idioms can legitimately reach the same row. **Grading note:** three of the four options select Charlie's row (verified by running them); grade all three as correct. For a future form, flip the stem to "which does NOT select Charlie's row?"

A. `df.loc[2]`

✓ *Also works*: returns the same row as a Series — accepted for credit.

B. `df.loc['Charlie']`

× `KeyError` — 'Charlie' is a *value* in the Name column, not an index label.

Q44 — pandas .loc — Answer

Review

`.loc` selects rows by *label* — an index label, a label-plus-columns pair, or a boolean mask — so several idioms can legitimately reach the same row. **Grading note:** three of the four options select Charlie's row (verified by running them); grade all three as correct. For a future form, flip the stem to "which does NOT select Charlie's row?"

A. `df.loc[2]`

✓ *Also works*: returns the same row as a Series — accepted for credit.

B. `df.loc['Charlie']`

× `KeyError` — 'Charlie' is a *value* in the Name column, not an index label.

C. `df.loc[df['Name'] == 'Charlie']`

✓ *Also works*: boolean mask selects Charlie without knowing the index (1-row DataFrame) — accepted for credit.

Q44 — pandas .loc — Answer

Review

`.loc` selects rows by *label* — an index label, a label-plus-columns pair, or a boolean mask — so several idioms can legitimately reach the same row. **Grading note:** three of the four options select Charlie's row (verified by running them); grade all three as correct. For a future form, flip the stem to "which does NOT select Charlie's row?"

A. `df.loc[2]`

✓ *Also works*: returns the same row as a Series — accepted for credit.

B. `df.loc['Charlie']`

× `KeyError` — 'Charlie' is a *value* in the Name column, not an index label.

C. `df.loc[df['Name'] == 'Charlie']`

✓ *Also works*: boolean mask selects Charlie without knowing the index (1-row DataFrame) — accepted for credit.

D. `df.loc[2, :]`

✓ The keyed answer: explicit "row label 2, all columns".

Q45 — GridSearchCV argument

Fill in the blank to specify the hyperparameter grid correctly:

```
from sklearn.svm import SVC
from sklearn.model_selection import GridSearchCV

# Initialize the SVM classifier
svm_classifier = SVC()

# Define the hyperparameter grid
param_grid = {
    'kernel': ['linear', 'rbf', 'poly', 'sigmoid'],
    'C': [0.1, 1, 10, 100],
    'gamma': [1, 0.1, 0.01, 0.001]
}

# Create the GridSearchCV object
grid_search = GridSearchCV(svm_classifier, __________, cv=5)
```

A. `param_grid`

B. `grid`

C. `hyperparameters`

D. `parameters`

Q45 — GridSearchCV argument — Answer

Review

GridSearchCV fits every combination in the grid ($4 \times 4 \times 4 = 64$ settings) with 5-fold cross-validation and keeps the best.

Q45 — GridSearchCV argument — Answer

Review

GridSearchCV fits every combination in the grid ($4 \times 4 \times 4 = 64$ settings) with 5-fold cross-validation and keeps the best.

✓ **Answer: A.** `param_grid` is the dictionary defined five lines up: `GridSearchCV(estimator, param_grid, cv=5)`. The other three names were never defined — each is a `NameError`.

Q46 — Essential preparation step

When preparing a dataset for logistic regression to classify items into two groups (e.g., real vs. fake), which of the following steps is essential?

A. Converting numeric features into categorical features.

B. Normalizing all text data using NLP techniques.

C. Splitting the dataset into training and testing subsets.

D. Aggregating all features into a single composite score.

Q46 — Essential preparation step — Answer

A. Converting numeric features into categorical features.

× Numeric → categorical usually *throws away* information.

Q46 — Essential preparation step — Answer

A. Converting numeric features into categorical features.

× Numeric → categorical usually *throws away* information.

B. Normalizing all text data using NLP techniques.

× There is no text in this dataset — NLP does not apply.

Q46 — Essential preparation step — Answer

A. Converting numeric features into categorical features.

× Numeric → categorical usually *throws away* information.

B. Normalizing all text data using NLP techniques.

× There is no text in this dataset — NLP does not apply.

C. Splitting the dataset into training and testing subsets.

✓ Without a held-out test set you cannot measure generalization — essential for *any* supervised model.

Q46 — Essential preparation step — Answer

A. Converting numeric features into categorical features.

× Numeric → categorical usually *throws away* information.

B. Normalizing all text data using NLP techniques.

× There is no text in this dataset — NLP does not apply.

C. Splitting the dataset into training and testing subsets.

✓ Without a held-out test set you cannot measure generalization — essential for *any* supervised model.

D. Aggregating all features into a single composite score.

× Aggregating features into one score destroys the inputs the model needs.

Q47 — Target variable

In a binary classification problem like distinguishing between two classes (e.g., real vs. fake), what is typically the role of the target variable?

A. To provide a continuous score representing the condition of each item.

B. To indicate the specific category or class to which each item belongs.

C. To list all possible categories for items in the dataset.

D. To summarize the features of the items into a single descriptor.

Q47 — Target variable — Answer

A. To provide a continuous score representing the condition of each item.

× A continuous condition score would be a *regression* target.

Q47 — Target variable — Answer

A. To provide a continuous score representing the condition of each item.

× A continuous condition score would be a *regression* target.

B. To indicate the specific category or class to which each item belongs.

✓ The target is the per-example ground truth: which class each item belongs to (0/1).

Q47 — Target variable — Answer

A. To provide a continuous score representing the condition of each item.

× A continuous condition score would be a *regression* target.

B. To indicate the specific category or class to which each item belongs.

✓ The target is the per-example ground truth: which class each item belongs to (0/1).

C. To list all possible categories for items in the dataset.

× The list of possible classes is metadata — the variable holds *each item's* class.

Q47 — Target variable — Answer

A. To provide a continuous score representing the condition of each item.

× A continuous condition score would be a *regression* target.

B. To indicate the specific category or class to which each item belongs.

✓ The target is the per-example ground truth: which class each item belongs to (0/1).

C. To list all possible categories for items in the dataset.

× The list of possible classes is metadata — the variable holds *each item's* class.

D. To summarize the features of the items into a single descriptor.

× Summarizing features is dimensionality reduction / feature engineering — not the target's role.

Q48 — Prediction loop

What is the main reason for using a loop to make predictions on a test set in machine learning?

A. To enhance the model's performance with each new prediction.

B. To apply the prediction function to each individual test sample and collect the results.

C. To decrease the likelihood of overfitting by evaluating multiple subsets.

D. To recalibrate the model parameters before making each prediction.

Q48 — Prediction loop — Answer

A. To enhance the model's performance with each new prediction.

× Prediction never improves the model — there is no learning at test time.

Q48 — Prediction loop — Answer

A. To enhance the model's performance with each new prediction.

× Prediction never improves the model — there is no learning at test time.

B. To apply the prediction function to each individual test sample and collect the results.

✓ A frozen model applied to each sample, outputs collected — that is all a prediction loop does.

Q48 — Prediction loop — Answer

A. To enhance the model's performance with each new prediction.

× Prediction never improves the model — there is no learning at test time.

B. To apply the prediction function to each individual test sample and collect the results.

✓ A frozen model applied to each sample, outputs collected — that is all a prediction loop does.

C. To decrease the likelihood of overfitting by evaluating multiple subsets.

× Overfitting is about *training*; looping at test time cannot change it.

Q48 — Prediction loop — Answer

A. To enhance the model's performance with each new prediction.

× Prediction never improves the model — there is no learning at test time.

B. To apply the prediction function to each individual test sample and collect the results.

✓ A frozen model applied to each sample, outputs collected — that is all a prediction loop does.

C. To decrease the likelihood of overfitting by evaluating multiple subsets.

× Overfitting is about *training*; looping at test time cannot change it.

D. To recalibrate the model parameters before making each prediction.

× Parameters are frozen at prediction time — recalibrating would be re-fitting (and must not happen).

Q49 — Conv output shape

An input image is 28×28. A convolutional layer applies a 3×3 kernel with stride 1 and no padding. What is the spatial size of the output feature map?

A. 28×28

B. 26×26

C. 25×25

D. 14×14

Q49 — Conv output shape — Answer

Review

Every distractor is a real formula misstep: 28×28 would need `padding='same'`; 25 is the off-by-one; 14 is what a 2×2 stride-2 pooling layer would give.

Q49 — Conv output shape — Answer

Review

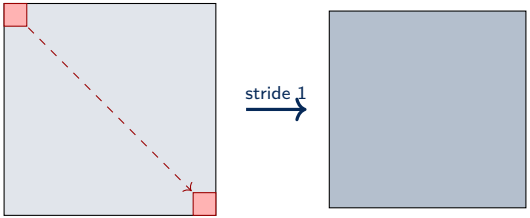
Every distractor is a real formula misstep: 28×28 would need `padding='same'`; 25 is the off-by-one; 14 is what a 2×2 stride-2 pooling layer would give.

✓ **Answer: B.** $\frac{N-K}{S}+1=\frac{28-3}{1}+1=26 \Rightarrow 26 \times 26$. A 3×3 window cannot hang off the edge, so each axis loses $K-1=2$ pixels.

Demo: counting valid kernel positions

$$\frac{N-K}{S}+1=\frac{28-3}{1}+1=26$$

3×3 kernel slides over all valid positions



28×28 input 26×26 output

Q50 — Kernel + activation

What does specifying a kernel size of 3×3 and an activation function in a convolutional layer typically achieve?

A. It determines the compactness of the model's representation and the threshold for activation.

B. It sets the filter dimensions for feature detection and applies a nonlinear transformation to the feature maps.

C. It controls the rate of learning and the depth of the network's architecture.

D. It aligns the data inputs with the expected outputs by resizing the feature maps.

Q50 — Kernel + activation — Answer

Review

The activation is there for the same reason as in Q31: without a non-linearity, stacked conv layers would collapse into a single linear operation.

Q50 — Kernel + activation — Answer

Review

The activation is there for the same reason as in Q31: without a non-linearity, stacked conv layers would collapse into a single linear operation.

A. It determines the compactness of the model's representation and the threshold for activation.

× "Compactness" and "activation threshold" — neither is what these two settings control.

Q50 — Kernel + activation — Answer

Review

The activation is there for the same reason as in Q31: without a non-linearity, stacked conv layers would collapse into a single linear operation.

A. It determines the compactness of the model's representation and the threshold for activation.

× "Compactness" and "activation threshold" — neither is what these two settings control.

B. It sets the filter dimensions for feature detection and applies a nonlinear transformation to the feature maps.

✓ Kernel size fixes the window the filter scans; the activation applies the non-linearity to the resulting maps.

Q50 — Kernel + activation — Answer

Review

The activation is there for the same reason as in Q31: without a non-linearity, stacked conv layers would collapse into a single linear operation.

A. It determines the compactness of the model's representation and the threshold for activation.

× "Compactness" and "activation threshold" — neither is what these two settings control.

B. It sets the filter dimensions for feature detection and applies a nonlinear transformation to the feature maps.

✓ Kernel size fixes the window the filter scans; the activation applies the non-linearity to the resulting maps.

C. It controls the rate of learning and the depth of the network's architecture.

× Learning rate belongs to the *optimizer*; depth to the architecture.

Q50 — Kernel + activation — Answer

Review

The activation is there for the same reason as in Q31: without a non-linearity, stacked conv layers would collapse into a single linear operation.

A. It determines the compactness of the model's representation and the threshold for activation.

× "Compactness" and "activation threshold" — neither is what these two settings control.

B. It sets the filter dimensions for feature detection and applies a nonlinear transformation to the feature maps.

✓ Kernel size fixes the window the filter scans; the activation applies the non-linearity to the resulting maps.

C. It controls the rate of learning and the depth of the network's architecture.

× Learning rate belongs to the *optimizer*; depth to the architecture.

D. It aligns the data inputs with the expected outputs by resizing the feature maps.

× Neither setting resizes or aligns anything.

Q51 — Hidden layer components

In constructing a basic feedforward neural network manually for predicting temperature, what are the essential components that need to be defined for the hidden layer?

A. Hidden weights and biases, used to calculate the output from the input layer before applying an activation function.

B. Hidden activation function only, as weights and biases are automatically adjusted.

C. Output weights and biases, as these are crucial for transferring data from the input to the hidden layer.

D. The learning rate and loss function, which are critical for the initial setup of each layer.

Q51 — Hidden layer components — Answer

A. Hidden weights and biases, used to calculate the output from the input layer before applying an activation function.

✓ Each layer's forward pass is $a = \phi(Wx + b)$: the layer's own W and b must be defined.

Q51 — Hidden layer components — Answer

A. Hidden weights and biases, used to calculate the output from the input layer before applying an activation function.

✓ Each layer's forward pass is $a = \phi(Wx + b)$: the layer's own W and b must be defined.

B. Hidden activation function only, as weights and biases are automatically adjusted.

× Weights are *updated* automatically by training — but they must be *defined* first.

Q51 — Hidden layer components — Answer

A. Hidden weights and biases, used to calculate the output from the input layer before applying an activation function.

✓ Each layer's forward pass is $a = \phi(Wx + b)$: the layer's own W and b must be defined.

B. Hidden activation function only, as weights and biases are automatically adjusted.

× Weights are *updated* automatically by training — but they must be *defined* first.

C. Output weights and biases, as these are crucial for transferring data from the input to the hidden layer.

× Output weights belong to the *output* layer.

Q51 — Hidden layer components — Answer

A. Hidden weights and biases, used to calculate the output from the input layer before applying an activation function.

✓ Each layer's forward pass is $a = \phi(Wx + b)$: the layer's own W and b must be defined.

B. Hidden activation function only, as weights and biases are automatically adjusted.

× Weights are *updated* automatically by training — but they must be *defined* first.

C. Output weights and biases, as these are crucial for transferring data from the input to the hidden layer.

× Output weights belong to the *output* layer.

D. The learning rate and loss function, which are critical for the initial setup of each layer.

× Learning rate and loss belong to the *training loop*, not to a layer.

Q52 — Coding: GridSearchCV

Complete the blanks below to enable the code to run successfully.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.svm import SVC
# Load the Iris dataset
data = load_iris()
X, y = data.data, data.target
# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
# Initialize the SVM classifier
svm_classifier = SVC()
# Define the hyperparameter grid for GridSearchCV
param_grid = {
    'kernel': ['linear', 'rbf', 'poly', 'sigmoid'],
    'C': [0.1, 1, 10, 100],
    'gamma': [1, 0.1, 0.01, 0.001]
}
# Create the GridSearchCV object with cross-validation (e.g., 5-fold cross-validation)
grid_search = GridSearchCV(__________ , ____________, cv=_____)
# Perform the hyperparameter tuning on the training data
grid_search.fit(X_train, y_train)
# Get the best hyperparameters from the GridSearchCV
best_params = grid_search.best_params_
print("Best hyperparameters:", best_params)
```

Q52 — Coding: GridSearchCV — Answer

Review

Positional order matters — estimator first, then the parameter grid; `cv=5` runs 5-fold cross-validation on each of the $4 \times 4 \times 4 = 64$ combinations.

Q52 — Coding: GridSearchCV — Answer

Review

Positional order matters — estimator first, then the parameter grid; `cv=5` runs 5-fold cross-validation on each of the $4 \times 4 \times 4 = 64$ combinations.

```
grid_search = GridSearchCV(svm_classifier, param_grid, cv=5)
```

Q53 — Coding: normal equation

Please complete code by replacing all `### EXERCISE CODE HERE` with your code.

```
import pandas as pd
import numpy as np
# weight-height dataset
dataset = pd.read_csv("https://gist.githubusercontent.com/nstokoe/.../weight-height.csv")
gender_mapping = {'Male': 0, 'Female': 1}
dataset["Gender"] = dataset["Gender"].map(gender_mapping)
display(dataset)
#put the columns of ones into the front of the dataset
dataset.insert(0, 'intercept', 1) ### EXERCISE CODE HERE
X = dataset[ ['intercept','Gender','Height'] ].values
y = dataset["Weight"].values
# Calculating the coefficients using the normal equation: (X^T * X)^(-1) * X^T * y
coefficients = np.linalg.inv(X.T @ X) @ X.T @ y ### EXERCISE CODE HERE
print(coefficients)
"""predict the weight of someone with height of 70 and who have gender 'male'"""
X_test = np.array([1,0,70])
# Making predictions for the test set
y_prediction = X_test @ coefficients
print(y_prediction)
```

Q53 — Coding: normal equation — Answer

Review

Blank 1: `dataset.insert(0, 'intercept', 1)` adds the column of ones so the intercept β_0 can be learned as an ordinary coefficient. Blank 2 is the normal equation itself, $\hat{\beta} = (X^\top X)^{-1} X^\top y$, written as `np.linalg.inv(X.T @ X) @ X.T @ y` — the closed-form solution to least squares; no gradient descent needed. **Instructor's note:** the live form displays both exercise lines *already completed* (followed by the `###` marker), so students effectively received this solution in the question text — blank those two lines in the Form before the next run.

Q53 — Coding: normal equation — Answer

Review

Blank 1: `dataset.insert(0, 'intercept', 1)` adds the column of ones so the intercept β_0 can be learned as an ordinary coefficient. Blank 2 is the normal equation itself, $\hat{\beta} = (X^\top X)^{-1} X^\top y$, written as `np.linalg.inv(X.T @ X) @ X.T @ y` — the closed-form solution to least squares; no gradient descent needed. **Instructor's note:** the live form displays both exercise lines *already completed* (followed by the `###` marker), so students effectively received this solution in the question text — blank those two lines in the Form before the next run.

```
dataset.insert(0, 'intercept', 1)

coefficients = np.linalg.inv(X.T @ X) @ X.T @ y
```

Q54 — Coding: logistic regression

Apply logistic regression to the following dataset, to classify bank notes as real or fake.

```
#The Banknote Authentication Dataset is a collection of data used for binary classification. It contains features
      extracted from images of genuine and counterfeit banknotes, captured using various sensors.

import pandas as pd
dataset = pd.read_csv("https://raw.githubusercontent.com/Dataweekends/zero_to_deep_learning_video/master/data/
      banknotes.csv")
display(dataset)

X = dataset.drop(columns=['class'])  # Replace 'target_column_name' with the actual target column name
y = dataset['class']
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
X_train, X_test, y_train, y_test = X_train.to_numpy(), X_test.to_numpy(), y_train.to_numpy(), y_test.to_numpy()

### EXERCISE CODE HERE
```

Q54 — Coding: logistic regression — Answer

Review

Fit on the training split only, evaluate on the held-out test split — that is the point of the split made above, and a solution that scores on training data misses it.

Q54 — Coding: logistic regression — Answer

Review

Fit on the training split only, evaluate on the held-out test split — that is the point of the split made above, and a solution that scores on training data misses it.

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

model = LogisticRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
```

Q55 — Coding: feedforward

Write a feedforward function that works on neural networks of 5 layers.

```
def feedforward(a, biases, weights):
    #layer 1 --> 2
    b = biases[0]
    w = weights[0]
    a_layer2 = sigmoid(np.dot(w, a) + b)

    # EXERCISE code
```

Q55 — Coding: feedforward — Answer

Review

Five layers means *four* weight/bias pairs (transitions between consecutive layers). Each step repeats the same pattern: $a \leftarrow \sigma(Wa + b)$, feeding each layer's activation into the next. The loop version generalizes to any depth — the insight is that a deep network is just this one line applied repeatedly.

Q55 — Coding: feedforward — Answer

Review

Five layers means *four* weight/bias pairs (transitions between consecutive layers). Each step repeats the same pattern: $a \leftarrow \sigma(Wa + b)$, feeding each layer's activation into the next. The loop version generalizes to any depth — the insight is that a deep network is just this one line applied repeatedly.

```
def feedforward(a, biases, weights):
    a = sigmoid(np.dot(weights[0], a) + biases[0])   # layer 1 -> 2
    a = sigmoid(np.dot(weights[1], a) + biases[1])   # layer 2 -> 3
    a = sigmoid(np.dot(weights[2], a) + biases[2])   # layer 3 -> 4
    a = sigmoid(np.dot(weights[3], a) + biases[3])   # layer 4 -> 5
    return a

# or, equivalently:
def feedforward(a, biases, weights):
    for w, b in zip(weights, biases):
        a = sigmoid(np.dot(w, a) + b)
    return a
```

Q56 — Coding: Keras Conv2D

You are building a simple CNN using TensorFlow's Keras API to classify images. Fill in the blank to add a convolutional layer to the model:

```
import tensorflow as tf

model = tf.keras.Sequential()

# Add a convolutional layer with 32 filters, a kernel size
# of (3,3), and 'relu' activation:
model.add(tf.keras.layers._____(32, (3, 3), activation='relu'))
```

Q56 — Coding: Keras Conv2D — Answer

Review

`Conv2D` is Keras's 2-D convolution layer: 32 filters, each 3×3, ReLU applied to the resulting feature maps. (`Conv1D` is for sequences, `Conv3D` for volumes; `Dense` would flatten away the spatial structure this layer exists to exploit.)

Q56 — Coding: Keras Conv2D — Answer

Review

`Conv2D` is Keras's 2-D convolution layer: 32 filters, each 3×3, ReLU applied to the resulting feature maps. (`Conv1D` is for sequences, `Conv3D` for volumes; `Dense` would flatten away the spatial structure this layer exists to exploit.)

```
model.add(tf.keras.layers.Conv2D(32, (3, 3), activation='relu'))
```